



Proximity-based explainable anomaly detection for time-series data with calibration, leakage safety

Ebubekir İnan 

Kayseri University, Faculty of Engineering, Architecture and Design, Department of Engineering Basic Sciences, Kayseri, 38000, Türkiye

ARTICLE INFO

2000 MSC:

62M10

62H30

68T05

Keywords:

Time series anomaly detection

Proximity-based modeling

Adaptive epsilon (rolling MAD)

Explainable AI (SHAP)

Semi-supervised learning

Deployment-oriented evaluation

ABSTRACT

Anomaly detection in non-stationary time-series streams is difficult when events are rare, score distributions drift, and operators can inspect only a fixed fraction of alarms. We introduce a deployment-oriented framework built on an unsupervised, train-referenced proximity score computed in a compact Principal Component Analysis (PCA) embedding and stabilized by a causal rolling Median Absolute Deviation (MAD) scale. The score is converted into decision-ready outputs through a strictly leakage-safe protocol: per-series temporal splitting, train-only isotonic calibration on a calibration-fit block, and deterministic tie-safe top- k selection to enforce an explicit alert budget (e.g., $q=1\%$) on a held-out evaluation block. Across three public labeled benchmarks—the Numenta Anomaly Benchmark (NAB), the Server Machine Dataset (SMD), and the UCR Time Series Anomaly Archive (UCR)—we report pooled pointwise area under the receiver operating characteristic curve (ROC-AUC) and area under the precision-recall curve (PR-AUC), together with fixed-budget precision and recall at an alert fraction $q = 1\%$, and series-aware bootstrap confidence intervals. The method achieves the best PR-AUC on SMD (0.500) and NAB (0.359) under the leak-free protocol, while remaining computationally feasible compared with forecasting-residual baselines. On UCR, results indicate an explicit trade-off on shape-driven anomalies, motivating operating-policy tuning. For the unlabeled Intel Berkeley Lab telemetry, we make no detection-accuracy claims; instead, we evaluate budgeted screening utility via a Composite Triage Score that summarizes coverage, stability, concentration, event duration, and runtime.

1. Introduction

We study time-series anomaly detection under non-stationarity, extreme class imbalance, and operational alarm budgets, where deployment requires calibrated anomaly probabilities and leakage-safe evaluation. Anomaly detection in time series is central to reliable decision making in finance, energy, and large-scale Internet of Things (IoT), where rare events must be surfaced under non-stationarity, noise, and severe class imbalance [1,2]. Community benchmarks such as the Numenta Anomaly Benchmark (NAB) [3–6] and open telemetry corpora (e.g., Intel Berkeley Lab [7]) typify these deployment realities: evolving regimes, incomplete or delayed supervision, and strong constraints on latency and memory. NAB supports streaming evaluation with labeled anomalies (and event-window conventions), whereas Intel-scale telemetry stresses long-duration operation in the presence of missingness and limited labeling—together enabling assessment under controlled scoring as well as deployment-like conditions.

A wide spectrum of detectors has been proposed. Classical baselines—Local Outlier Factor (LOF) [8], one-class support vector machine (SVM) [9], and Isolation Forest [10]—operationalize density, support, or isolation principles and remain indispensable ref-

E-mail address: ebubekirinan@kayseri.edu.tr

<https://doi.org/10.1016/j.iot.2026.101920>

Received 10 October 2025; Received in revised form 8 February 2026; Accepted 5 March 2026

Available online 10 March 2026

2542-6605/© 2026 Elsevier B.V. All rights are reserved, including those for text and data mining, AI training, and similar technologies.

erence points. Distance- and density-based clustering extensions [11] improve flexibility in heterogeneous data. Deep learning further expanded the modeling space through sequence and reconstruction paradigms: long short-term memory (LSTM) encoder–decoders [12], convolutional forecasters (DeepAnT) [13], and variational autoencoder (VAE)-based seasonal key performance indicator (KPI) modeling [14] established representation learning as a strong alternative [15,16], followed by stochastic recurrent networks (OmniAnomaly) [17], hybrid autoencoders (USAD) [18], and Transformer-based detectors [19] that capture multi-scale and non-local temporal structure. Recent work has emphasized robustness to contaminated training data [20], temporal-relationship graphs and adaptive smoothing [21], industrial deployments with graph attention under strict latency budgets [22], retentive network-based anomaly detection in cyber-physical systems [23], and lightweight federated pipelines for industrial control systems (ICS)/IoT environments [24]. Application-driven studies further confirm practical impact in manufacturing and optimization contexts [25,26], and domain-grounded pipelines report explainable performance on mission-critical telemetry [27]. Surveys consolidate these directions and highlight persistent gaps at the intersection of accuracy, scalability, and interpretability in multivariate streaming settings [28–31].

Despite this progress, deployment-grade anomaly detection under drift and extreme rarity still faces recurring obstacles. First, when positives are scarce, receiver operating characteristic (ROC)-based summaries can be overly optimistic; precision–recall measures provide a more faithful characterization of operational utility [32,33]. Second, drift and heavy-tailed noise induce score instability; robust dispersion estimators such as the (rolling) median absolute deviation (MAD) can stabilize scaling, yet they remain under-integrated in end-to-end anomaly systems [34]. Third, decision-making is often budgeted (e.g., a fixed fraction of points can be reviewed), but many detectors output uncalibrated scores rather than decision-ready probabilities. While isotonic regression is a principled monotone post-hoc calibrator [35,36], anomaly pipelines rarely integrate calibration under a strictly leak-free, train-only protocol where the calibrator and the operating point are learned without peeking at the final reporting block. Finally, although explainability has advanced from post-hoc attributions (e.g., Shapley Additive exPlanations (SHAP)) [37] toward more intrinsically interpretable designs [38,39], practitioners still need explanations that align with the detector’s native evidence and remain stable under drift and heterogeneous regimes.

This work is positioned around a deployment-oriented objective: capturing time-series anomalies under drift and extreme imbalance in compute-constrained settings via a train-referenced adaptive score, rolling-MAD stabilization, and leak-free decision making. In this paper, *proximity* is used in the data-mining sense to denote *train-referenced neighborhood deviation cues*—distance-based departure, density-oriented irregularity, and temporal-change signals—that are transformed into actionable anomaly scores and calibrated probabilities. We do *not* assume or invoke a proximity relation in the axiomatic/topological sense; the term is strictly descriptive of the neighborhood evidence extracted relative to the training reference.

Throughout the paper, *Proposed* refers to the *unsupervised* detection core: (i) the proximity-based anomaly score and (ii) *train-only* calibration learned strictly on `calib_fit`. Any supervised model used for interpretability (e.g., a Light Gradient-Boosting Machine (LightGBM) head with SHAP) is an *optional analysis layer* and is not required to produce the reported detection results.

Building on the drift-resistant behavior of MAD scaling [34], we introduce an integrated framework that closes practical deployment gaps without increasing model complexity:

- **Train-referenced adaptive anomaly scoring:** a train-referenced evidence layer that computes complementary normalized cues (level departure, a density cue with a safe fallback, velocity in embedding-increment space, and prototype distance) and fuses them into a label-agnostic score $s(t) \in [0, 1]$ under an adaptive rolling-MAD scale to improve stability under non-stationarity while remaining parameter-light.
- **Leak-free calibration and budgeted decision policy:** on labeled benchmarks, isotonic regression is fitted *only* on a calibration-fit block to map $s(t)$ to calibrated probabilities, and a fixed-budget operating threshold (e.g., top-1%) is learned on the same calibration-fit block and transferred unchanged to a held-out calibration-evaluation block for final reporting [35].
- **Actionable explainability and deployment triage:** On labeled benchmarks, SHAP attributions are computed on the leak-free evaluation block (`calib_eval`) for the deployed decision score, i.e., the isotonic-calibrated anomaly probability [37]. On the unlabeled Intel Berkeley Lab telemetry, we do not claim supervised detection performance; instead, we report deployment-triage utility under a fixed alert budget using label-free distributional normalization, Empirical Cumulative Distribution Function (ECDF) / Chunked Empirical Cumulative Distribution Function (ChECDF) and operational criteria (coverage, stability, concentration, and event characteristics).

Operationally, the framework targets strong performance under precision–recall evaluation [32,33] and budget-tempered alarm regimes, while supporting an accuracy–runtime trade-off that remains sustainable relative to compute-heavy forecasting baselines [19]. Given the rare-event nature of anomalies, we emphasize PR-AUC and budget-aligned precision/recall metrics; ROC-AUC is reported for completeness but can be optimistic under severe imbalance.

Across complementary settings, the approach exhibits robust behavior under the stated leak-free protocol. On NAB [3–6], we observe strong PR-oriented ranking and conservative fixed-budget alerting (e.g., PR-AUC = 0.359 with PR@1% = 1.000 under point-wise pooled reporting on `calib_eval`), while maintaining low runtime relative to forecasting residual baselines. On Server Machine Dataset (SMD), the proposed method achieves strong PR-AUC (= 0.500) and competitive budgeted performance, with explicit trade-offs against fast classical baselines. On UCR at scale, absolute PR values remain small due to extreme sparsity and heterogeneity, yet the proposed method yields favorable fixed-budget behavior with tractable runtime. On Intel-scale streams [7], where labels are absent, the method produces stable hotspot rankings and interpretable alert patterns under a fixed budget, summarized by a Composite Triage Score constructed from coverage, stability, concentration, event duration, and runtime. Ablations indicate that rolling-MAD stabilization improves temporal stability and that the decision layer reduces operating-point volatility under severe imbalance.

The remainder of the paper details the proximity-based scoring and rolling-MAD stabilization, the strictly leak-free calibration and budgeted thresholding protocol, and the explainability and triage pipelines, followed by comprehensive experiments, ablations, and a discussion of deployment considerations and limitations.

2. Method: Adaptive proximity framework

The proposed framework outputs pointwise anomaly scores using an unsupervised proximity core and converts them to calibrated anomaly probabilities using a calibrator fit on `calib_fit` only. A separate supervised *head* (LightGBM) is introduced solely to enable feature attribution (SHAP) and is kept outside the detection pipeline used in all quantitative comparisons.

2.1. Overview of the proposed framework

We propose a deployment-oriented anomaly detection framework for non-stationary time series that targets three practical constraints: (i) severe class imbalance, (ii) regime shifts and drift, and (iii) the need for transparent, actionable outputs under limited compute budgets. The framework is designed to operate in both labeled benchmarks and unlabeled operational telemetry. Its core output is a per-time-point anomaly score, which can be further converted into calibrated anomaly probabilities when labels are available, and into ranked event candidates and stream-level triage summaries when labels are absent.

Pipeline at a glance: For each time series (or sensor stream) processed independently, the framework follows the steps below:

1. **Causal preprocessing (leak-free):** Numeric features are forward-filled with a bounded limit and missing values are imputed using train-only medians.
2. **Train-only embedding:** A compact representation is learned on the training prefix via standardization and PCA, yielding a fixed three-dimensional embedding per time index.
3. **Proximity evidence cues:** Four complementary proximity cues are computed relative to the training reference: (a) drift-adaptive level departure, (b) a density cue via train-fitted outlier scoring (with a safe fallback), (c) a velocity cue in embedding-increment space, and (d) distance to a robust training prototype.
4. **Fusion into a single score:** The train-normalized evidence cues are aggregated by a monotone fusion operator to form a label-agnostic anomaly score $s(t) \in [0, 1]$.
5. **Calibration and decision policy (labeled setting):** Using a strictly leak-free temporal split, isotonic regression is fitted on a calibration-fit block to obtain calibrated probabilities, and a budgeted threshold is learned on calibration-fit and applied to a held-out calibration-evaluation block.
6. **Deployment triage (unlabeled setting):** On unlabeled telemetry, the same score is used to extract candidate events and compute stream-level triage statistics (coverage, stability, concentration, and duration) without any label dependence.

A complete, implementation-faithful pseudocode of Steps 1–6 is provided in [Appendix A \(Algorithms 1–3\)](#).

All operations that involve fitting—including imputation statistics, scaling, PCA, optional density modeling, calibration, and threshold selection—are learned exclusively from the designated training or calibration-fit segments and are never informed by future samples. Each series is processed independently, enabling a manifest/parts workflow that bounds memory usage and supports large corpora.

Consider a single series indexed by $t \in X = \{1, \dots, n\}$. Let $X_{tr} \subset X$ denote the leak-free training prefix used to fit preprocessing and the embedding. The train-fitted embedding is denoted by $\mathbf{z}_t \in \mathbb{R}^3$. The proximity layer produces four train-normalized evidence cues in $[0, 1]$, which are fused into $s(t)$. In labeled settings, $s(t)$ is mapped to calibrated probabilities $\hat{p}(t)$ and thresholded under a fixed budget policy.

To mitigate regime shifts, we rescale scores with a rolling MAD computed causally (no future access), ensuring stability under drift while preserving anomaly ranking. Rolling-MAD scaling provides a robust, causal mechanism to adapt proximity to drift. Train-referenced neighborhoods ensure that all proximity cues are comparable across time while remaining leak-free. Finally, the framework prioritizes operational feasibility: it yields competitive accuracy under extreme imbalance while maintaining predictable runtime and resilience to long-job interruptions, which is critical when contrasting with heavy deep forecasting baselines.

2.2. Leak-free embedding and adaptive reference scale

We construct a per-series proximity layer that is strictly leak-free and deployment-aligned. Each observation at time index t is treated as a singleton set $\{t\}$, and proximity is evaluated *relative to a training reference* within the same series. Let $X = \{1, \dots, n\}$ index the time-ordered samples of a single stream and let $X_{tr} \subset X$ denote the leak-free training prefix used to fit all preprocessing and representation steps. We begin from a numeric feature map $\Phi(t) \in \mathbb{R}^p$ (base statistical features and robust transforms) and enforce causality via forward-fill with a bounded limit, followed by train-median imputation; both operations are computed without using future samples.

The concrete causal feature construction used to form $\Phi(t)$ is summarized in [Algorithm 1](#), while the leak-free sanitization, train-median imputation, train-only standardization, and PCA embedding steps are given in [Algorithm 2](#).

All representation learning is fitted on X_{tr} only. Specifically, we apply a train-only standardization and then a train-only PCA map $\Psi : \mathbb{R}^p \rightarrow \mathbb{R}^3$ to obtain a compact embedding

$$\mathbf{z}_t := \Psi(\Phi(t)) \in \mathbb{R}^3, \quad t \in X, \quad (1)$$

where we denote $\mathbf{z}_t = (\phi_1(t), \phi_2(t), \phi_3(t))$. This three-dimensional embedding is intentionally fixed across datasets to keep the proximity layer lightweight and comparable across streams, while avoiding unstable high-dimensional density estimation.

To stabilize proximity under drift, we introduce a time-local scale using a causal rolling median absolute deviation (MAD) computed on the first embedding coordinate ϕ_1 . For a window length W , define

$$\text{MAD}_W(t) := \text{median}\left(\left|\phi_1(u) - \text{median}(\phi_1(u') : u' \in \mathcal{W}_t)\right| : u \in \mathcal{W}_t\right), \quad (2)$$

where $\mathcal{W}_t \subseteq \{1, \dots, t\}$ is the causal window (with the same minimum-period rule as in the released implementation). The adaptive scale is

$$\epsilon_t := k_{\text{MAD}} \cdot \text{MAD}_W(t), \quad \epsilon_t \geq \epsilon_{\min} > 0, \quad (3)$$

where $k_{\text{MAD}} > 0$ controls the scale (default and tuned settings are reported in Section 4.3) and $\epsilon_{\min}$ is enforced numerically for stability. Missing MAD values are filled by a train-only fallback statistic to preserve leak-freedom.

Algorithm 2 provides the exact leak-free computation of the adaptive causal scale ϵ_t (including train-only fallback handling for missing MAD values).

2.3. Proximity evidence cues: Level, density cue, velocity, and prototype distance

The proximity component reacts to local deviations, whereas the density component penalizes sustained departures from typical neighborhoods, improving robustness under noise and gradual drift. The proximity layer yields four complementary evidence maps in $[0, 1]$ per time index t , each computed in a train-referenced manner. The end-to-end construction of the proximity evidence cues (including train-referenced kNN distances, optional HDBSCAN with a leak-free fallback, and train-only min-max normalization) is summarized in Algorithm 2.

Let $k \in \mathbb{N}$ be the neighborhood size. For any $t \in X$, define the mean k NN distance from $\mathbf{z}_t$ to the *training* embedding $\{\mathbf{z}_s : s \in X_{\text{tr}}\}$ by

$$d_k(t \mid X_{\text{tr}}) := \frac{1}{k} \sum_{s \in \text{kNN}(\mathbf{z}_t; X_{\text{tr}})} \|\mathbf{z}_t - \mathbf{z}_s\|_2. \quad (4)$$

All subsequent evidence cues are computed from train-referenced distances and normalized using train-only statistics, ensuring leak-freedom and making scores comparable over time within each stream.

Level evidence cue prox_level_mu: We quantify level departure relative to the training reference set via an adaptively scaled distance transform. Using the rolling scale ϵ_t , we define the raw cue

$$\tilde{\mu}_{\text{level}}(t) := 1 - \exp\left(-\frac{d_k(t \mid X_{\text{tr}})}{\epsilon_t + \delta}\right), \quad \delta > 0, \quad (5)$$

where $d_k(t \mid X_{\text{tr}}) \geq 0$ is a train-referenced distance quantity and δ ensures numerical stability. Since the mapping $x \mapsto 1 - e^{-x}$ is bounded in $[0, 1]$ for $x \geq 0$, $\tilde{\mu}_{\text{level}}(t)$ is intrinsically bounded. We then apply *train-only* min-max normalization

$$\mu_{\text{level}}(t) := \text{MinMax}_{X_{\text{tr}}}(\tilde{\mu}_{\text{level}}(t)) \in [0, 1], \quad (6)$$

which guarantees leak-freedom and yields a comparable scale over time within a stream. Intuitively, $\mu_{\text{level}}(t)$ increases as t becomes more atypical with respect to the training reference set, while ϵ_t adapts the sensitivity to regime-dependent dispersion.

Density evidence cue prox_level_outlier: To complement distance-based departure with a density-oriented signal, we compute a train-fitted outlier cue on the embedding. When feasible (sufficient training length and availability), HDBSCAN is fitted on $\{\mathbf{z}_s : s \in X_{\text{tr}}\}$ and yields outlier scores $o_{\text{HDB}}(t)$ for $t \in X_{\text{tr}}$. Implementation details for the density cue and the leak-free kNN fallback policy are provided in Algorithm 2. We define the raw outlier cue

$$\tilde{\mu}_{\text{out}}(t) := \begin{cases} o_{\text{dens}}(\mathbf{z}_t), & \text{if a train-fitted density scorer is feasible,} \\ d_k(t \mid X_{\text{tr}}), & \text{otherwise (fallback),} \end{cases} \quad (7)$$

and again apply train-only min-max normalization

$$\mu_{\text{out}}(t) := \text{MinMax}_{X_{\text{tr}}}(\tilde{\mu}_{\text{out}}(t)) \in [0, 1]. \quad (8)$$

This construction remains robust under short series or constrained environments, since it degrades gracefully to the k NN distance proxy without affecting leak-freedom.

Velocity evidence cue prox_velocity_mu: Abrupt changes are often captured more reliably in the derivative space. We define the embedding increment $\Delta \mathbf{z}_t := \mathbf{z}_t - \mathbf{z}_{t-1}$ for $t \geq 2$ and $\Delta \mathbf{z}_1 := \mathbf{0}$. Using the same train-referenced neighborhood principle, we compute

$$d_k^\Delta(t \mid X_{\text{tr}}) := \frac{1}{k} \sum_{s \in \text{kNN}(\Delta \mathbf{z}_t; \Delta X_{\text{tr}})} \|\Delta \mathbf{z}_t - \Delta \mathbf{z}_s\|_2, \quad \Delta X_{\text{tr}} := \{\Delta \mathbf{z}_s : s \in X_{\text{tr}}\}, \quad (9)$$

and set

$$\mu_{\text{vel}}(t) := \text{MinMax}_{X_{\text{tr}}}(d_k^\Delta(t \mid X_{\text{tr}})) \in [0, 1]. \quad (10)$$

Prototype-distance evidence cue `proto_dist_1`: Finally, we measure departure from a robust training prototype. Let

$$\mathbf{p} := \text{median}\{\mathbf{z}_s : s \in X_{\text{tr}}\} \in \mathbb{R}^3 \quad (11)$$

be the coordinate-wise median prototype in the embedding. Define

$$d_{\text{proto}}(t) := \|\mathbf{z}_t - \mathbf{p}\|_2, \quad \mu_{\text{proto}}(t) := \text{MinMax}_{X_{\text{tr}}}(d_{\text{proto}}(t)) \in [0, 1]. \quad (12)$$

The prototype cue is particularly stable under heavy imbalance, since it relies on a robust summary of the training reference rather than on labels or event frequency.

2.4. Proximity fusion and score formation

The four evidence cues $\mu_{\text{level}}(t)$, $\mu_{\text{out}}(t)$, $\mu_{\text{vel}}(t)$, and $\mu_{\text{proto}}(t)$ characterize complementary aspects of deviation from the training reference: geometric level departure, density irregularity, dynamical disruption, and prototype distance. For each series, we produce a pointwise anomaly score $s(t)$ using only information available up to time t , yielding a stream-compatible detector. We combine them into a single proximity score via a fusion operator

$$s(t) := \text{Fuse}\left(\mu_{\text{level}}(t), \mu_{\text{out}}(t), \mu_{\text{vel}}(t), \mu_{\text{proto}}(t)\right) \in [0, 1]. \quad (13)$$

In the released framework, $\text{Fuse}(\cdot)$ is implemented as a small family of monotone aggregators to support ablation and deployment constraints. Examples include arithmetic mean, top-2 mean, max, noisy-or, softmax pooling with temperature parameter, p -norm pooling, and an inverse-MAD weighted fusion that emphasizes cues computed under locally reliable scale. Since all inputs are already train-normalized to $[0, 1]$, fusion is numerically stable and comparable across time within a stream.

The output $s(t)$ is an *unsupervised* anomaly score: it does not use labels and can be computed on fully unlabeled telemetry. In labeled settings, $s(t)$ is subsequently calibrated and thresholded using leak-free splits (Section 2.5). In unlabeled deployment triage, $s(t)$ can be used directly to produce ranked event candidates and stream-level summaries (Section 2.7).

2.5. Leak-free calibration and threshold selection

To obtain well-behaved anomaly probabilities suitable for operational decision-making, we apply a strictly leak-free calibration and thresholding protocol. For each series, data are split into three chronological blocks: a training block (used for feature fitting and the proximity layer), a calibration-fit block, and a calibration-evaluation block. The proximity layer defined in Sections 2.2-2.3 is computed using train-fitted preprocessing only; labels are not used in proximity construction. The leakage-safe calibration and operating-point learning protocol (train-only / `calib_fit`-only fitting, isotonic calibration, deterministic exact top- k budget selection, and strict transfer to `calib_eval`) is detailed in Algorithm 3.

On labeled datasets, calibration is fitted using the calibration-fit block and evaluated on the held-out calibration-evaluation block. We employ isotonic regression as a non-parametric, monotone calibrator mapping raw scores $s(t)$ to calibrated probabilities $\hat{p}(t) \in [0, 1]$. All calibration fitting is performed *only* on calibration-fit data, and performance metrics are reported *only* on calibration-evaluation data to avoid optimistic bias.

Thresholds are learned on calibration-fit under a fixed budget policy. Concretely, for a target rate q (e.g., $q = 1\%$), we set $k_{\text{fit}} = \lceil q |\text{calib_fit}| \rceil$ and select the exact top- k_{fit} scores on `calib_fit` using deterministic tie-safe selection. We record the implied threshold $\tau_{\text{fit}}(q)$ (the minimum score within the selected set) for reporting (Thr@fit). For final reporting on `calib_eval`, we do not re-estimate any threshold; instead, we transfer the budget k_{fit} and select exactly k_{fit} alarms on `calib_eval` using the same tie-safe rule. This yields directly actionable fixed-budget operating metrics (PR@1% and R@1%) without using any information from `calib_eval` to set the budget.

2.6. Computational profile, scalability, and practical baselines

A central design goal is deployment feasibility under resource constraints and long-running streams. The proximity layer operates per series with train-only fitted preprocessing, and its dominant costs are: (i) PCA transform in $\mathbb{R}^p \rightarrow \mathbb{R}^3$, (ii) repeated k NN queries against the training embedding, and (iii) optional HDBSCAN fitting on the training embedding. All components are lightweight relative to deep sequence models and can be executed with bounded memory via per-series processing and manifest-based streaming.

Transformer-based forecasting residual baselines (e.g., Informer, PatchTST, iTransformer) can be competitive in settings where substantial compute budgets and stable execution environments are available. However, under strict runtime and connectivity constraints, they may become impractical: training and inference times can exceed interactive execution budgets, and long jobs are vulnerable to session interruption. For this reason, we include transformer baselines in a *budgeted* configuration (bounded steps / early stopping), and we interpret their results jointly with their runtime. This reporting aligns with deployment priorities: methods that do not reliably finish within the operational budget cannot be assumed available in real-time monitoring.

To ensure fair comparison, all baselines are evaluated under the same leak-free splitting and threshold-learning protocol as the proposed method. In addition to classical unsupervised detectors (e.g., Isolation Forest, LOF, COPOD) and time-series residual baselines (e.g., ETS residuals, Matrix Profile), we report ablations that isolate the contribution of the proximity layer (e.g., base statistical features only) and analyze sensitivity to the proximity hyperparameters (W , k_{MAD} , min_cluster_size) and fusion choice. This combination of accuracy, calibration, and runtime evidence supports a deployment-oriented conclusion: the proposed proximity framework offers a strong accuracy–efficiency trade-off across labeled benchmarks and remains usable on unlabeled telemetry streams.

2.7. Unlabeled deployment triage (Intel)

The Intel Berkeley telemetry is largely unlabeled; therefore, we do *not* claim detection accuracy on Intel. Instead, we position Intel as an *unlabeled deployment triage* setting: under a fixed alert budget, the goal is to prioritize nodes and time periods for inspection using risk scores that are operationally useful—broadly covering the fleet, sufficiently stable under calibration transfer, not overly concentrated on a few nodes, producing actionable (non-degenerate) alert durations, and remaining computationally feasible.

We apply a time-ordered 3-way split `train` / `calib_fit` / `calib_eval` per node. All calibration mappings (ECDF/ChECDF) are fitted *only* on `calib_fit` and then applied unchanged to `calib_eval`. To reflect alert-rate constraints, we learn a budgeted threshold on `calib_fit` using a deterministic, tie-safe top- k rule at $q = 1\%$:

$$k_{\text{fit}} = \lceil q N_{\text{fit}} \rceil, \quad \tau(q) = \text{the score at rank } k_{\text{fit}} \text{ on } \text{calib_fit}.$$

The learned $\tau(q)$ is then applied *unchanged* to `calib_eval`, ensuring that no information from `calib_eval` influences threshold selection. For the proposed family, proximity channels are *not* recomputed on Intel in this triage block; we only fuse the precomputed proximity scores and calibrate them using `calib_fit`. The unlabeled triage transfer protocol (label-free normalization fit on `calib_fit`, unchanged application to `calib_eval`, and fixed-budget top- k alerting) follows [Algorithm 3](#).

We summarize each method by a Composite Triage Score (CTS) constructed from five deployment-oriented criteria. Let $\mathcal{N}$ be the set of nodes and let A_i be the alerted points on node i in `calib_eval` (points with score $\geq \tau(q)$).

1. **Coverage (C)**: fraction of nodes exhibiting at least one alert,

$$C = \frac{1}{|\mathcal{N}|} \sum_{i \in \mathcal{N}} \mathbf{1}\{|A_i| > 0\}.$$

2. **Stability (S)**: rank-consistency of node-level risk between `calib_fit` and `calib_eval`. Let $\bar{s}_i^{\text{cf}}$ and $\bar{s}_i^{\text{ce}}$ be the mean calibrated score on node i in `calib_fit` and `calib_eval`, respectively. We compute Spearman correlation across nodes:

$$S = \rho_{\text{Spearman}}\left(\{\bar{s}_i^{\text{cf}}\}_{i \in \mathcal{N}}, \{\bar{s}_i^{\text{ce}}\}_{i \in \mathcal{N}}\right).$$

3. **Concentration (H)**: Herfindahl–Hirschman Index (HHI) of alert mass across nodes. Let $p_i = \frac{|A_i|}{\sum_{j \in \mathcal{N}} |A_j| + \epsilon}$, then

$$HHI = \sum_{i \in \mathcal{N}} p_i^2,$$

where lower values indicate a more evenly distributed alert allocation.

4. **Duration (D)**: mean alert-event duration on `calib_eval`. An *event* is a maximal contiguous segment of points with score $\geq \tau(q)$. We report the mean event duration in minutes, using dataset timestamps (or the known sampling interval when timestamps are regular).
5. **Runtime (R)**: wall-clock inference time for producing calibrated risk scores on `calib_eval` (seconds).

To combine heterogeneous units, each component is min–max normalized across the compared methods. For criteria to be maximized (Coverage, Stability), we use

$$\text{mm}(x) = \frac{x - \min(x)}{\max(x) - \min(x) + \epsilon}.$$

For criteria to be minimized (HHI, Duration, Runtime), we use the inverted min–max transform

$$\text{mm}^\downarrow(x) = \frac{\max(x) - x}{\max(x) - \min(x) + \epsilon}.$$

We then define

$$CTS = \frac{1}{5} \left(\text{mm}(C) + \text{mm}(S) + \text{mm}^\downarrow(HHI) + \text{mm}^\downarrow(D) + \text{mm}^\downarrow(R) \right).$$

CTS is a *triage* score: it ranks methods by operational screening utility under an alert budget; it is not a substitute for accuracy on labeled benchmarks.

Results and operational implications are reported in [Section 4.4](#).

2.8. Complexity and practical deployment notes

This section characterizes the computational and memory footprint of the proposed pipeline and summarizes implementation choices that matter for deployment. We report asymptotic costs in terms of generic problem sizes (no dataset-specific timings).

Consider one stream of length n with a training prefix X_{tr} of size $n_{\text{tr}} = |X_{\text{tr}}|$. Let $\Phi(t) \in \mathbb{R}^p$ denote the per-time feature map after causal preprocessing, $d = 3$ the fixed embedding dimension, k the neighborhood size for k NN queries, and W the causal window size used for rolling-MAD stabilization (and, where applicable, rolling feature computations).

Feature computation and causal preprocessing: Most base features used in $\Phi(\cdot)$ are rolling statistics and robust transforms computed causally. With standard streaming implementations, rolling means/medians/variances and robust z -style transforms are

$O(np)$ time overall, with $O(Wp)$ working memory for window buffers (or equivalent summaries). Forward-fill with a bounded limit is linear in n , and train-median imputation and standardization require one pass over X_{tr} , i.e., $O(n_{tr}p)$ time and $O(p)$ state. The causal rolling-feature and robust-transform construction referenced here is specified in [Algorithm 1](#).

Train-only embedding (standardization and PCA): All representation learning is fitted on X_{tr} only. Standardization is $O(n_{tr}p)$. For PCA with fixed $d=3$, a truncated/SVD-style fit scales as $O(n_{tr}pd)$ in typical implementations and stores $O(pd)$ parameters for the projection. Applying the learned projection to the full stream is linear, $O(npd)$.

Rolling-MAD stabilization (causal scale): The adaptive scale ϵ_t uses a causal window $\mathcal{W}_t \subseteq \{1, \dots, t\}$ of size at most W . Regardless of the specific rolling-MAD implementation, the update depends only on past values and therefore preserves causality. The online state is $O(W)$ (or an equivalent compact summary) for the selected embedding coordinate(s).

Train-referenced neighborhood queries (dominant term): The proximity evidence cues in [Sections 2.3-2.4](#) rely on k NN distances from (i) z_t to the *training* embedding $\{z_s : s \in X_{tr}\}$ and (ii) Δz_t to the *training* increment set ΔX_{tr} . With naive exact search, each query compares against n_{tr} reference points, giving per-stream time

$$O(n n_{tr} d)$$

for each k NN-based evidence cue, plus $O(n_{tr}d)$ memory to store the reference set (and similarly for increments). In practice, exact or approximate nearest-neighbor indexing can reduce average query time; the framework is agnostic to the index choice and remains leak-free as long as the index is built on X_{tr} only.

Optional density cue and safe fallback: The density/outlier cue is trained on the training embedding only ([Section 2.3](#)). Its cost is incurred once per stream (train-only) and is therefore amortized over n timestamps. When the density model is infeasible (e.g., insufficient n_{tr} or constrained environments), the method falls back to a train-referenced k NN-distance proxy. This guarantees that the pipeline remains executable without changing the leak-free protocol.

Normalization, fusion, and score formation: Train-only min-max normalization of evidence-cue maps requires computing train statistics and is linear in n_{tr} per map. Fusion into the final score $s(t) \in [0, 1]$ is pointwise and constant time, $O(1)$ per timestamp (hence $O(n)$ per stream), and does not introduce additional fitting.

Calibration and budgeted operating point: On labeled benchmarks, isotonic calibration is fitted on `calib_fit` only. If $n_{cf} = |\text{calib_fit}|$, isotonic fitting is dominated by sorting and costs $O(n_{cf} \log n_{cf})$ time with $O(n_{cf})$ memory. Learning the budgeted threshold at rate q via deterministic tie-safe top- k selection can be implemented in $O(n_{cf})$ expected time (selection) or $O(n_{cf} \log n_{cf})$ (sorting), and is applied unchanged to `calib_eval`. On unlabeled Intel triage, the same threshold-transfer mechanism is used, and CTS components are computed by linear-time aggregates over alerted points and contiguous events. These calibration and budgeted operating-point steps correspond to [Algorithm 3](#).

Memory profile and streaming alignment: The pipeline is series-local and can be executed independently per stream. The dominant persistent state per stream consists of: (i) preprocessing statistics $O(p)$, (ii) PCA parameters $O(pd)$, (iii) training reference sets in embedding and increment spaces $O(n_{tr}d)$, and (iv) rolling buffers $O(Wp)$ (feature windows) and $O(W)$ (MAD scale). This structure supports a manifest/parts workflow that processes one stream (or a small batch) at a time, bounding peak memory and allowing straightforward parallelization across streams.

Deployment options (fixed reference vs. controlled refresh): The experiments in this paper use a fixed training reference per stream to ensure strict comparability under a leak-free protocol. In operational settings, one may optionally refresh the reference set (and re-fit the embedding) on a controlled schedule, provided that updates are restricted to past data and are not influenced by the evaluation horizon. Similarly, calibration can be updated when labels become available, without changing the underlying unsupervised scoring mechanism.

The computational cost is dominated by train-referenced neighborhood queries; the remaining components (rolling features, PCA transform, normalization, fusion, calibration, and budgeted thresholding) are lightweight and naturally streaming-compatible. This design matches the deployment goal of producing actionable, transparent scores under constrained compute, while maintaining a strictly leak-free train/calibration/evaluation separation.

3. Experimental protocol

3.1. Datasets

To stress-test deployment-oriented anomaly detection under non-stationarity and extreme class imbalance, we select four datasets that are deliberately heterogeneous in (i) labeling availability (fully labeled vs. unlabeled), (ii) dimensionality (univariate vs. multivariate telemetry), (iii) annotation semantics (pointwise vs. interval/event conventions), (iv) scale (from $\sim 10^5$ to $> 10^7$ points), and (v) sampling irregularities and missingness. This design prevents over-fitting the narrative to a single benchmark and enables a unified evaluation of (a) ranking quality under imbalance (PR-AUC), (b) budgeted alerting regimes (PR@1%, R@1%), (c) leakage-safe calibration, and (d) compute sustainability under realistic runtime constraints, as summarized in [Table 1](#).

3.1.1. NAB (Labeled, univariate; pointwise vs. event semantics)

NAB is a streaming benchmark designed for real-time anomaly detection where concept drift and imbalance are prominent [\[5\]](#). We use 13 canonical NAB streams spanning artificial and real sources (e.g., ambient temperature, request latency, taxi demand, traffic, and social volume). In our processed subset, the pooled table evaluates pointwise labels on the leak-free `calib_eval` segment; we

Table 1
Datasets used in this study (Dataset-level statistics from our processed splits).

Dataset	Streams/Nodes	Dim.	Labeled?	Scale (rows)
NAB (subset)	13	1	Yes	98,288
SMD	28	38	Yes	1,416,825 (train + test)
UCR (full)	250	1	Yes	≈15,911,730
Intel Lab	54	4	No	2,303,286

explicitly note that NAB also supports event/window-based scoring and treat it as a complementary view rather than a replacement for pointwise, budgeted alerting.

3.1.2. SMD (Labeled, multivariate server telemetry)

SMD provides multivariate server-machine telemetry with a fixed train/test split and dense labels on the test partition [40]. This dataset is a primary “multivariate deployment” vignette: it couples correlated metrics ($D = 38$) with non-trivial anomaly structure, enabling evaluation of proximity-based fusion under cross-sensor interactions. In our loader, 28 machine streams are available, with 708,405 train rows, 708,420 test rows, and 29,444 anomalous test points.

SMD provides an official train/test split, where labels are available for the test partition only. To keep calibration and operating-point selection strictly out-of-sample while using labels in a deployment-aligned way, we instantiate our leak-free three-way regime *within the official test stream* in chronological order: an initialization prefix (`test_train_prefix`, 70%) for fitting the score-related components that require a reference, a calibration-fit block (`calib_fit`, 15%) for isotonic calibration and budgeted threshold learning, and a held-out calibration-evaluation block (`calib_eval`, 15%) used *only* for final reporting. Crucially, we do not assume that the initialization prefix is perfectly anomaly-free; this mirrors real deployments where early telemetry may already contain incidents. Leak-freedom is ensured by design because no information from `calib_eval` is used in fitting, calibration, or threshold selection.

3.1.3. UCR time series anomaly archive (Labeled; scale and heterogeneity stress test)

The UCR anomaly archive aggregates heterogeneous univariate series with interval-style anomaly annotations, spanning multiple domains and lengths [41,42]. It functions as a scale and generalization stress test: in our full run, the processed archive contains 250 series and ≈15.9M points with severe imbalance (total positives ≈45,614). This setting is intentionally hostile to compute-heavy baselines, therefore highlighting Pareto trade-offs among ranking quality, budget metrics, and runtime.

3.1.4. Intel berkeley research lab (Unlabeled; deployment triage)

Intel Lab sensor telemetry provides long-duration, real-world multivariate measurements collected from a sensor network (temperature, humidity, light, voltage) with irregular sampling and non-negligible missingness [7]. As the corpus is unlabeled, we do not claim supervised accuracy; instead, we use it to validate whether calibrated, budgeted scoring produces operationally plausible alert patterns (coverage, stability, concentration, event statistics) under realistic deployment artifacts. Our processed version contains 54 nodes, 2.30M rows, and missingness that is concentrated in the `value/voltage` channel.

On the Intel Berkeley Lab corpus, ground-truth anomaly labels are not available; therefore, we do not perform probability calibration in the supervised sense (i.e., mapping scores to calibrated class probabilities against labels). Instead, we apply *label-free distributional normalization* learned on `calib_fit` and transferred to `calib_eval` to obtain a *normalized risk score* that is comparable across sensors and time. Concretely, we fit an empirical CDF (ECDF) or its chunked variant (ChECDF) on `calib_fit` and transform raw anomaly scores into quantile-scale values in $[0, 1]$. This transformation preserves the within-stream ranking while stabilizing thresholds under drift and sensor-specific score distributions, enabling deployment-style triage without claiming calibrated probabilities.

3.2. Experimental coverage

We evaluate a deployment-oriented anomaly detection framework across three labeled benchmarks and one unlabeled deployment triage corpus, with method families selected to reflect practical operating constraints (drift, severe class imbalance, and compute limits). On the labeled benchmarks (NAB, SMD, and UCR), we report results for:

- (i) the proposed adaptive proximity scoring pipeline (adaptive proximity score with rolling-MAD stabilization and train-only calibration),
- (ii) representative classical outlier detectors (Isolation Forest, LOF, COPOD), (iii) residual-based forecasting baselines (ETS residual), and
- (iv) a subsequence/shape-based baseline (Matrix Profile).

Transformer forecaster residual baselines (Informer, iTransformer, and PatchTST) are included where computationally feasible (NAB and SMD), reflecting realistic deployment budgets; for the large-scale UCR benchmark we restrict the comparison set to methods that remain tractable under the same evaluation regime, and we additionally include a statistical-only ablation to isolate the contribution of the proximity components. For the unlabeled Intel Berkeley Lab sensor corpus, we do not claim supervised detection performance; instead, we perform deployment triage under a fixed alert budget using label-free distributional normalization (ECDF/ChECDF) fitted on `calib_fit` and transferred unchanged to `calib_eval`, and we report operational statistics (coverage, stability, concentration, and event characteristics) for multiple ProxFuse variants alongside representative classical scorers.

3.3. Leakage-safe splitting and evaluation regime

Reliable anomaly detection under drift and extreme class imbalance is highly sensitive to evaluation leakage, particularly when thresholds and calibrators are tuned on information that overlaps with the reported test segment. To ensure a strictly out-of-sample assessment, we adopt a *leakage-safe, temporally ordered, per-series* evaluation regime that explicitly separates (i) representation/score learning, (ii) calibration and operating-point selection, and (iii) final reporting.

Temporal splitting (per series): For each time series, samples are ordered chronologically (timestamp for NAB/Intel; integer index t for UCR; test-time index t for SMD). We then construct a *three-way* split that respects time:

Train : 70% Calib : 15% Eval : 15%,

implemented as contiguous blocks to prevent any look-ahead. This per-series splitting avoids cross-series leakage and preserves realistic deployment conditions where decisions are made forward in time.

Calibration block: calib_fit vs. calib_eval: The 15% calibration segment is further treated as an explicit decision layer. In our main [Table 5](#) protocol, we distinguish:

- **calib_fit:** used *only* to (a) fit the probability calibrator and (b) learn the operating policy under a fixed alert budget (e.g., top-1%).
- **calib_eval:** used *only* for final metric reporting.

This separation prevents optimistic bias that would arise if the same calibration data were used both to choose the operating point and to report performance.

Train-only calibration (leak-free):

Our core detector produces a label-agnostic anomaly score $s(t)$ by fusing train-referenced proximity evidence cues with robust, train-only temporal normalization. On labeled benchmarks (e.g., NAB, SMD), we optionally train a lightweight supervised mapping on **calib_fit** (LightGBM) to combine the feature set—including the proximity-derived components and $s(t)$ —into a single decision score. This supervised head is used *only* when labels are available and is trained strictly within the leak-free split; it is then calibrated (e.g., isotonic) on **calib_fit** and evaluated on the held-out **calib_eval**. For unlabeled Intel streams, the supervised head is disabled and the pipeline operates purely on $s(t)$ followed by label-free normalization (ECDF/ChECDF) for deployment-style triage. Accordingly, SHAP explanations are reported for the calibrated output of the supervised head on labeled datasets, ensuring that interpretability is computed on the same score used for decisions.

When a method yields real-valued anomaly scores or raw probabilities, we map them to calibrated anomaly probabilities via isotonic regression fit on **calib_fit** only. Concretely, let $s(\cdot)$ denote the method score on **calib_fit** and $y \in \{0, 1\}$ the corresponding labels (available for labeled benchmarks). We fit an isotonic map g on (s, y) and apply g to **calib_eval** without refitting. If **calib_fit** contains a single class (ill-posed calibration), we fall back to the identity mapping, preserving strict leak-freeness and avoiding degenerate fits.

All detection metrics are computed on the deployed decision score: (i) on labeled datasets, ROC/PR and fixed-budget selection are evaluated on the calibrated core decision score $p_b(t) = g(s(t))$, where the isotonic calibrator g is fitted on **calib_fit** and applied unchanged to **calib_eval**; (ii) on unlabeled Intel streams, the evaluation uses the core score with label-free normalization (ECDF/ChECDF) for deployment-style triage. The supervised LightGBM head is used only as an optional surrogate for SHAP attributions and does not affect any reported detection metric.

Budget-aware operating point learned on calib_fit: Deployment commonly constrains the alert rate. We therefore learn the operating policy under a fixed alert budget q (default $q = 1\%$) on **calib_fit** and apply it unchanged to **calib_eval**. Specifically, we compute

$$k_{\text{fit}} = \lceil q \cdot |\text{calib_fit}| \rceil,$$

select the exact top- k_{fit} scores on **calib_fit** with deterministic tie handling, and record the implied threshold τ_{fit} (the minimum score within the selected set). Here, τ_{fit} is reported only as a diagnostic summary of the operating regime on **calib_fit**; the deployed decision rule on **calib_eval** is fixed-budget selection by transferring k_{fit} (exact top- k_{fit} with tie-safe determinism). On **calib_eval**, we then evaluate PR@1% and R@1% by selecting *exactly* k_{fit} alarms (tie-safe exact top- k). This procedure yields a faithful, policy-aligned estimate of fixed-budget alerting performance and avoids unstable thresholding in the presence of ties or heavy score quantization.

Pooled reporting and uncertainty: For each dataset, we report pointwise pooled ROC-AUC and PR-AUC on **calib_eval**, along with the fixed-budget operating metrics PR@1% and R@1%. Confidence intervals (CI_{95}) are computed via *series-aware bootstrap*, resampling complete series (not individual points) to respect temporal dependence and heterogeneous stream characteristics. Where relevant, we also report paired delta CI_{95} for method comparisons using matched bootstrap resamples, yielding uncertainty on performance *differences* without inflating optimism through pointwise resampling.

Dataset-specific instantiations (controlled differences): The above protocol is applied uniformly across labeled benchmarks, with only dataset-imposed adaptations:

- **NAB (labeled, univariate):** evaluation is reported as *pointwise pooled* on **calib_eval**. Since NAB also provides event/window-based scoring, we explicitly distinguish our pointwise [Table 5](#) protocol from NAB's event scoring to avoid conflation of objectives.
- **SMD (labeled, multivariate):** labels are provided only for the official *test* partition. Therefore, to preserve the same leak-free separation between (i) reference fitting, (ii) calibration/threshold learning, and (iii) final reporting, we apply an internal, time-ordered 70/15/15 split *within the official test stream* (`test_train_prefix/calib_fit /calib_eval`). We do not require the

initialization prefix to be anomaly-free; leak-freedom is enforced by restricting all fitting and decision selection to `calib_fit` and reporting only on `calib_eval`.

- **UCR (labeled, large-scale):** each series is ordered by integer index t and split into contiguous 70/15/15 blocks; budget k_{fit} is learned on `calib_fit` and applied to `calib_eval` using exact top- k selection.
- **Intel Berkeley Lab (unlabeled):** supervised metrics are not available; we therefore retain the same fixed-budget alerting policy (top-1%) and report deployment triage statistics (coverage, stability, concentration, and event characteristics) rather than ROC/PR.

By (i) enforcing temporal, per-series splitting, (ii) separating `calib_fit` from `calib_eval`, and (iii) learning fixed-budget operating points exclusively on `calib_fit`, the evaluation isolates genuine generalization under drift and imbalance. This protocol aligns the reported performance with realistic deployment constraints while preventing threshold- and calibration-induced leakage.

3.4. Baselines, compute budget, and reporting

We benchmark the proposed framework against baseline families that reflect deployment-realistic design choices: (i) compute-feasible classical detectors that score observations directly, and (ii) forecasting-residual baselines that capture recent forecaster-driven pipelines but typically require substantially higher wall-clock cost. All methods follow the leakage-safe evaluation regime in Section 3.3. In particular, any operating threshold (including budget-derived thresholds) is learned on `calib_fit` and applied unchanged to `calib_eval`.

Baselines: The classical detector suite consists of IsolationForest (IF), Local Outlier Factor in novelty mode (LOF), COPOD, ETS residual scoring, and MatrixProfile. This selection spans complementary mechanisms— isolation-based scoring, local density deviation, copula-based tail behavior, classical residual modeling, and shape-based discord discovery—and represents the methods most frequently considered when training and serving deep sequence models is impractical. To represent modern forecaster-driven detection, we include Informer, iTransformer, and PatchTST in a residual configuration: a forecaster is fit on the designated training portion, prediction residuals yield pointwise anomaly scores, and those scores are subsequently calibrated and thresholded using the same `calib_fit` $\rightarrow$ `calib_eval` protocol as all other methods.

Compute budget and fairness: To ensure controlled and reproducible comparisons, we enforce a fixed compute policy across methods. All runs are executed in the same environment and measured using wall-clock seconds with a consistent timing procedure. Forecasting baselines are evaluated under explicit compute limits (e.g., `max_steps` and associated early-stop/timeout rules) together with deterministic fail-safe behavior: if a forecaster exceeds the budget, diverges, or cannot be produced for a subset of series, the event is recorded and a predefined fallback scoring rule is applied for that subset so that evaluation remains complete and comparable. This policy is aligned with the manuscript's deployment focus: the goal is not unconstrained state-of-the-art training, but an accuracy–cost comparison under realistic resource limits.

Accordingly, we treat the forecasting-residual models as compute-bounded references rather than unconstrained state-of-the-art systems, and we position the comparison around compute-feasible baselines that are realistic for continuous monitoring deployments.

Metrics and operating policy: We report threshold-free ranking quality, fixed-budget operating behavior, wall-clock cost, and uncertainty estimates. As threshold-free metrics, we compute ROC-AUC and PR-AUC by sweeping a decision threshold τ over the deployed decision score (the calibrated probability). Let $y(t) \in \{0, 1\}$ denote the pointwise ground-truth label and let $\hat{y}_\tau(t) \in \{0, 1\}$ be the binary decision obtained by thresholding at level τ . On the reporting block (`calib_eval`), we form the confusion-matrix counts $TP(\tau), FP(\tau), TN(\tau), FN(\tau)$ by *pointwise pooling* over all timestamps (and all streams) in the dataset, and define

$$\text{Precision}(\tau) = \frac{TP(\tau)}{TP(\tau) + FP(\tau)}, \quad \text{Recall}(\tau) = \frac{TP(\tau)}{TP(\tau) + FN(\tau)}. [43]$$

The ROC curve plots $(FPR(\tau), TPR(\tau))$ with $TPR(\tau) = \text{Recall}(\tau)$ and $FPR(\tau) = \frac{FP(\tau)}{FP(\tau) + TN(\tau)}$, and ROC-AUC is the area under this curve [43]. The PR curve plots

$(\text{Recall}(\tau), \text{Precision}(\tau))$, and PR-AUC is the area under this curve. Given severe class imbalance, PR-AUC is treated as the primary metric throughout, while ROC-AUC is reported for completeness; PR-based summaries are generally more informative for operational utility in this regime [32,33].

Because operational monitoring constrains the alert rate, we additionally report PR@1% and R@1% under a fixed 1% alert budget. The budget is learned on `calib_fit` by selecting the exact top- $k_{\text{fit}} = \lceil 0.01 \cdot |\text{calib_fit}| \rceil$ alarms using tie-safe selection; the same k_{fit} is then applied on `calib_eval` to compute precision and recall at the operating point. We also report $\text{Thr@fit}(1\%)$, defined as the minimum score among the selected alarms on `calib_fit`, as an interpretable summary of the learned operating regime. For unlabeled Intel data, supervised detection metrics are unavailable; we therefore report fixed-budget triage statistics (coverage, stability, concentration, and event characteristics) under the same 1% alert policy.

Runtime: Runtime is reported as wall-clock seconds under the compute policy above and is used in the Pareto analysis in Section 4.2 to quantify the deployment-relevant accuracy–cost trade-off.

Uncertainty and paired comparisons: To quantify variability across heterogeneous streams, we compute CI_{95} using series-aware bootstrapping, resampling complete series rather than individual points. For selected comparisons, we also report paired delta CI_{95} by computing metric differences on matched bootstrap resamples (e.g., $\Delta = \text{metric}(\text{Proposed}) - \text{metric}(\text{Baseline})$), which directly characterizes uncertainty in performance gaps while preserving the per-series dependence structure, as summarized in Table 2.

3.5. Reproducibility and configuration summary

We summarize the experimental policy and the minimal set of configuration choices required to reproduce the reported results. Full run-time environment snapshots (including a `pip freeze`) are exported by the implementation and referenced as supplementary artifacts.

Leakage-safe evaluation policy: All datasets are processed per series in chronological order. We use a per-series contiguous three-way split with a training prefix ($\text{TRAIN_FRAC}=0.70$) and a remaining calibration block (30%), which is split into `calib_fit` and `calib_eval` using $\text{CALIB_FIT_FRAC}=0.50$ (i.e., a 70/15/15 split). Any calibrator and any operating-point selection rule (including budget-derived thresholds) are learned on `calib_fit` and applied unchanged to `calib_eval`. All preprocessing steps (imputation, scaling, PCA) are fitted on the training prefix only.

Fixed-budget operating point (1%) and deterministic selection: We enforce a fixed alert budget $q = \text{BUDGET_Q} = 0.01$. On `calib_fit`, we set $k_{\text{fit}} = \lceil q |\text{calib_fit}| \rceil$ and select the exact top- k_{fit} scores. Selection is deterministic and tie-safe: after computing $\text{thr} = \min\{s_i : i \in \text{Top-}k_{\text{fit}}\}$, we take all indices with $s_i > \text{thr}$ and fill the remaining quota from $s_i = \text{thr}$ in ascending index order. The learned k_{fit} is then transferred to `calib_eval` to report $\text{PR@1\%}$ and $\text{R@1\%}$.

Uncertainty (series-aware): CI_{95} values are computed via series-aware bootstrapping (resampling complete series with replacement). We use $\text{N_BOOT}=500$ replicates and report the $[0.025, 0.975]$ quantiles. Paired $\Delta\text{-CI}_{95}$ values are computed on matched bootstrap draws.

Run-time artifacts: The implementation exports a machine-readable environment snapshot (JSON) and a full `pip freeze` text file alongside the produced result tables (paths printed at run time). These artifacts provide the complete package inventory and enable exact replay of the experimental environment.

4. Main results

4.1. Labeled benchmarks: Main results

Tables 3–5 summarize the primary labeled-benchmark results under our leakage-safe protocol with train-only calibration and budget-aware thresholding. Across all benchmarks, we report both ranking-oriented metrics (ROC–AUC, PR–AUC) and operational, policy-driven metrics under a fixed alert budget ($\text{PR@1\%}$, $\text{R@1\%}$). This dual view is essential because, under severe class imbalance and non-stationarity, high ROC–AUC does not necessarily translate into deployable alerting performance at low budgets; conversely, budget metrics can reveal practical trade-offs that are invisible to global AUC measures.

Table 2

Minimal configuration summary (as implemented). Full environment snapshots and complete package lists are exported as supplementary artifacts.

Item	Setting
Randomness control	SEED=42; NumPy seed set; LightGBM random_state=42
NAB split policy	Per-series contiguous split; TRAIN_FRAC=0.70; calibration 30% split by CALIB_FIT_FRAC=0.50 into calib_fit/calib_eval (i.e., 70/15/15).
Alert budget	BUDGET_Q=0.01; exact top- k_{fit} on calib_fit; tie-safe deterministic selection
CI protocol	Series-aware bootstrap; N_BOOT=500; CI_{95} via quantiles (0.025, 0.975)
Causal feature windows (NAB)	Rolling stats at windows (5, 15, 30); robust z-score window $w=200$
Missing/Inf handling	Causal forward fill (FFILL_LIMIT=5); train-median imputation (NaN median $\rightarrow 0.0$); Inf/NaN sanitized to finite
Embedding	StandardScaler fit on train prefix; train-only PCA up to 3 components; padded to (ϕ_1, ϕ_2, ϕ_3)
Proximity parameters	DEFAULT: $k_{\text{MAD}}=1.4826$, $W=200$, $\text{min_cluster_size}=30$; TUNED: $k_{\text{MAD}}=2.0$, $W=250$, $\text{min_cluster_size}=40$
Neighborhood cue	Train-referenced k NN with $K_{\text{FOR_KNN}}=5$
Density cue	USE_HDBSCAN=True; trained on train embedding when feasible; otherwise leak-free k NN fallback
Supervised head	LightGBM + (optional) isotonic calibration fit on calib_fit only (identity if single-class)
Compute-bounded forecasters	If enabled: max_steps=100, INPUT_SIZE=64, WBS=32, LR=10 ⁻³ ; fail-soft fallback preserves completeness
Run-time snapshot (this study)	Google Colab; Python 3.12.12; Linux kernel 6.6.105+; RAM ≈ 12 GiB; no GPU detected; key libs: NumPy 2.0.2, Pandas 2.2.2, scikit-learn 1.6.1, LightGBM 4.6.0, statsmodels 0.14.6, STUMPY 1.13.0

SMD (multivariate, labeled): robust ranking with competitive budget behavior: On SMD (Table 3), the proposed Adaptive Proximity framework achieves the strongest precision–recall ranking performance ($PR-AUC = 0.500$), indicating improved ordering of anomalous points in an imbalanced multivariate setting. At the same time, classical outlier detectors (e.g., IsolationForest and COPOD) attain higher ROC–AUC and, in the strict 1% budget regime, IsolationForest yields higher $PR@1\%$ and $R@1\%$. We therefore avoid any claim of uniform dominance and instead emphasize the operational trade-off: our method prioritizes robust PR ranking while remaining competitive under fixed-budget alerting. Consistent with this interpretation, the confidence intervals indicate that improvements against compute-feasible forecasting baselines (e.g., Informer residual) are substantial, while differences to the strongest classical baselines may remain within uncertainty bounds and should be interpreted cautiously.

Table 3

SMD pooled-calibration (pointwise) results with leak-free evaluation. Top-1% budget is learned on `calib_fit` and evaluated on `calib_eval`. CI_{95} computed via series-aware bootstrapping.

Method	ROC–AUC	PR–AUC	PR@1%	R@1%	Thr@fit	k	Runtime(s)	CI_{95} (ROC/PR)	CI_{95} (PR@1/R@1)
Adaptive Proximity (Proposed)	0.871327	0.500174	0.623706	0.170525	0.985229	1063	483.611989	[0.722, 0.949] / [0.060, 0.813]	[0.021, 0.334] / [0.970] / [0.005, 0.334]
IsolationForest	0.896975	0.480147	0.938852	0.256687	0.612336	1063	43.717737	[0.820, 0.948] / [0.302, 0.729]	[0.467, 0.520] / [0.991] / [0.114, 0.520]
COPOD	0.907725	0.439436	0.757291	0.207047	0.786.853027	1063	192.291184	[0.845, 0.944] / [0.323, 0.590]	[0.488, 0.458] / [0.898] / [0.108, 0.458]
Ablation: Base Statistical Only	0.861571	0.379802	0.493885	0.135031	0.975787	1063	474.241205	[0.726, 0.942] / [0.052, 0.762]	[0.014, 0.305] / [0.975] / [0.003, 0.305]
LOF (novelty)	0.827215	0.144129	0.256820	0.070216	0.52.213554	1063	510.281173	[0.719, 0.919] / [0.084, 0.384]	[0.100, 0.218] / [0.529] / [0.025, 0.218]
Informer (forecast residual, max_steps = 100)	0.629989	0.053922	0.032926	0.009002	0.944912	1063	66.607002	[0.515, 0.772] / [0.032, 0.118]	[0.005, 0.050] / [0.154] / [0.001, 0.050]
MatrixProfile (m = 64)	0.600219	0.047779	0.011289	0.003086	8.444966	1063	71.065019	[0.399, 0.721] / [0.013, 0.116]	[0.000, 0.015] / [0.060] / [0.000, 0.015]
PatchTST (forecast residual, max_steps = 100)	0.479981	0.037337	0.031044	0.008488	0.957984	1063	3077.263019	[0.363, 0.615] / [0.023, 0.070]	[0.014, 0.038] / [0.096] / [0.003, 0.038]
iTransformer (forecast residual, max_steps = 100)	0.459508	0.035904	0.029163	0.007973	0.954249	1063	3151.234861	[0.345, 0.583] / [0.025, 0.071]	[0.019, 0.045] / [0.118] / [0.004, 0.045]
ETS-residual	0.367337	0.025970	0.001881	0.000514	7.538135	1063	208.953632	[0.239, 0.547] / [0.011, 0.051]	[0.000, 0.007] / [0.032] / [0.000, 0.007]

NAB (univariate, labeled): deployment-oriented performance under drift with strong precision at low budget: On NAB (Table 4), the proposed method again attains the highest $PR-AUC$ ($= 0.359$) under pointwise pooled evaluation on `calib_eval`, supporting the central claim that proximity-driven scoring with rolling robust stabilization is effective under drift and extreme imbalance. Notably, at a fixed 1% alert budget learned on `calib_fit`, the method produces very high precision ($PR@1\% = 1.000$) with non-trivial recall ($R@1\% = 0.061$), reflecting a conservative yet clean alerting regime suitable for deployment scenarios where false alarms are costly. Runtime remains orders-of-magnitude lower than heavy transformer forecasters (e.g., Informer residual) while providing stronger $PR-AUC$, although we stress that runtime should be interpreted under the stated compute policy and implementation details. Paired delta intervals further support a cautious reading: while the method is competitive against LOF in terms of PR measures, differences may not be uniformly significant across all metrics, motivating our later operating-curve analysis over a range of budgets.

UCR (large-scale, labeled) - scale and generalization with explicit trade-offs: UCR constitutes the most challenging generalization and scale test (Table 5), where absolute $PR-AUC$ values are small due to extreme sparsity and heterogeneous series characteristics. In this regime, the proposed method does not dominate ROC–AUC, and shape-based approaches such as MatrixProfile attain higher ROC–AUC. However, under the operational 1% alert budget, our method yields substantially higher $PR@1\%$ and $R@1\%$ than MatrixProfile, alongside a large runtime advantage over both MatrixProfile and LOF. Importantly, paired delta intervals confirm that LOF can outperform our method in ROC–AUC on UCR, and we report this explicitly; our contribution is therefore best characterized as a deployment-oriented Pareto point: improved budgeted alerting quality with tractable compute at scale, rather than universal superiority on all global metrics.

Summary of labeled-benchmark evidence: Taken together, the labeled results support the manuscript’s central positioning: the proposed proximity-based adaptive scoring, combined with rolling robust stabilization and leakage-safe calibration, yields strong PR ranking performance on two labeled benchmarks (SMD and NAB), and provides competitive, policy-relevant budget behavior with favorable runtime characteristics across datasets. Where classical or shape-based baselines excel on ROC–AUC or at specific operating points, we treat these as meaningful trade-offs rather than contradictions, and we analyze them explicitly via Pareto and budget-sweep curves in subsequent sections.

Finally, because real deployments often involve missing or delayed labels, we complement the labeled evaluations with an unlabeled deployment triage study on the Intel Berkeley Lab corpus (Table 6), reported separately to avoid over-claiming supervised performance in an unlabeled setting.

Table 4

NAB pooled-calibration (pointwise) results with leak-free evaluation. Per-series 70/30 split; the calibration block is split into `calib_fit/calib_eval`. A top-1% budget is learned on `calib_fit` (tie-safe exact top- k); on `calib_eval`, the deployed rule transfers k_{fit} (with τ_{fit} reported for diagnostics only). Metrics are reported on `calib_eval`. CI_{95} is computed via series-aware bootstrapping. Note: NAB also defines event/window-based scoring; this table reports pointwise pooled metrics.

Method	ROC-AUC	PR-AUC	PR@1%	R@1%	Thr@fit	k	Runtime(s)	CI_{95} (ROC/PR)	CI_{95} (PR@1/R@1)
Adaptive Proximity (Proposed)	0.636	0.359	1.000	0.061	0.5486	148	23.77	[0.507, 0.827] / [0.136, 0.646]	[0.247, 1.000] / [0.016, 0.108]
LOF (novelty)	0.655	0.266	0.466	0.029	3.1000	148	61.43	[0.561, 0.771] / [0.119, 0.398]	[0.166, 0.802] / [0.015, 0.049]
PatchTST (forecast residual, max_steps = 100)	0.552	0.208	0.230	0.014	9.7518e+03	148	355.04	[0.312, 0.772] / [0.066, 0.600]	[0.000, 1.000] / [0.000, 0.066]
ETS-residual	0.501	0.202	0.081	0.005	2.8527e+06	148	29.61	[0.338, 0.713] / [0.070, 0.281]	[0.000, 0.450] / [0.000, 0.041]
Ablation: Base Statistical Only	0.528	0.193	0.541	0.033	0.5284	148	15.54	[0.508, 0.557] / [0.117, 0.281]	[0.173, 0.919] / [0.009, 0.081]
iTransformer (forecast residual, max_steps = 100)	0.510	0.189	0.216	0.013	1.0344e+04	148	465.10	[0.285, 0.732] / [0.062, 0.573]	[0.000, 1.000] / [0.000, 0.066]
IsolationForest	0.521	0.183	0.176	0.011	0.1876	148	2.08	[0.330, 0.632] / [0.066, 0.343]	[0.079, 0.654] / [0.007, 0.049]
Informer (forecast residual, max_steps = 100) fallback_series = 1/13	0.484	0.176	0.318	0.019	1.1006e+04	148	6073.60	[0.229, 0.738] / [0.059, 0.385]	[0.000, 0.712] / [0.000, 0.047]
MatrixProfile (m = 64)	0.488	0.147	0.000	0.000	9.0147	148	2.22	[0.366, 0.601] / [0.065, 0.248]	[0.000, 0.022] / [0.000, 0.001]
COPOD	0.393	0.145	0.142	0.009	60.0168	148	2.88	[0.188, 0.579] / [0.053, 0.253]	[0.022, 0.438] / [0.002, 0.022]

Table 5

UCR pooled-calibration (pointwise) results with leak-free evaluation at large scale. Top-1% budget is learned on `calib_fit` and evaluated on `calib_eval`. CI_{95} computed via series-aware bootstrapping.

Method	ROC-AUC	PR-AUC	PR@1%	R@1%	Thr@fit	k	Runtime(s)	CI_{95} (ROC/PR)	CI_{95} (PR@1/R@1)
Adaptive Proximity (Proposed)	0.579138	0.016549	0.041939	0.099920	1.540493e-02	23,868	394.630502	[0.525, 0.646] / [0.008, 0.022]	[0.064] / [0.057, 0.145]
MatrixProfile (m = 64)	0.709964	0.009935	0.020823	0.049611	6.447163e+00	23,868	2186.026975	[0.658, 0.758] / [0.006, 0.007]	[0.036] / [0.017, 0.082]
LOF (novelty)	0.658774	0.009424	0.024594	0.058595	5.547092e-01	23,868	17890.948818	[0.597, 0.711] / [0.005, 0.009]	[0.045] / [0.028, 0.090]
COPOD	0.494981	0.004951	0.007039	0.016770	-1.231583e+01	23,868	106.348866	[0.377, 0.628] / [0.003, 0.003]	[0.012] / [0.009, 0.030]
IsolationForest	0.498687	0.004832	0.000000	0.000000	2.179272e-01	23,868	45.645328	[0.361, 0.610] / [0.002, 0.009]	[0.004] / [0.000, 0.008]
Ablation: Base Statistical Only	0.507773	0.004686	0.008128	0.019365	1.591403e-02	23,868	328.521006	[0.454, 0.571] / [0.003, 0.007]	[0.021] / [0.003, 0.050]
ETS-residual	0.477130	0.004064	0.000042	0.000100	1.981570e+06	23,868	519.279710	[0.352, 0.596] / [0.002, 0.008]	[0.000] / [0.000, 0.000]

4.2. Pareto analysis: Accuracy vs runtime

Beyond single-metric ranking, deployment-oriented anomaly detection requires explicit trade-off analysis between detection quality and wall-clock cost. To this end, we perform a Pareto analysis where each method is represented as a point with accuracy on the vertical axis and runtime on the horizontal axis. We use PR-AUC as the primary accuracy metric due to extreme class imbalance, and visualize runtime on a logarithmic scale (i.e., PR-AUC vs. $\log(\text{runtime})$) to account for the multi-order-of-magnitude spread induced by forecasting-based baselines. Runtime is reported as wall-clock seconds under the fixed compute policy described in [Section 4](#) (same environment and measurement procedure as in [Tables 3–5](#)). Wall-clock runtime is measured end-to-end under the fixed compute policy, including preprocessing, scoring, and post-processing (calibration/thresholding where applicable. A method is Pareto-dominated if there exists another method that achieves higher (or equal) PR-AUC at lower (or equal) runtime, with at least one strict improvement.

PR-AUC vs. runtime: deployment-sustainable accuracy: Across datasets, the proposed Adaptive Proximity framework consistently occupies a favorable region of the accuracy-runtime plane. On NAB ([Table 4](#)), the proposed method attains the highest PR-AUC while remaining orders-of-magnitude faster than transformer forecasters (Informer, iTransformer, PatchTST residual baselines), yielding a clear deployment-sustainable operating point when compute is constrained. On SMD ([Table 3](#)), the proposed method achieves the strongest PR-AUC, while lighter classical baselines such as IsolationForest provide a complementary point on the frontier by trading a small PR-AUC decrease for substantially reduced runtime. This pattern is consistent with the intended positioning of the proposed framework: shifting the Pareto frontier upward in PR quality without relying on compute-heavy forecasting backbones.

Table 6

Intel unlabeled triage: CTS ranking and key components under the leak-free $q = 1\%$ budget (threshold learned on `calib_fit` and applied unchanged to `calib_eval`). Component scores (CTS_C – CTS_R) are the min–max normalized terms used in CTS; HHI and D are reported in raw units for interpretability.

Method	CTS	CTS_C	CTS_S	CTS_H	CTS_D	CTS_R	HHI	D (min)	Runtime (s)
Proposed (invMAD-weighted) + ChECDF + ECDF	0.823952	0.792	0.777	0.930	0.668	1.000	0.108986	35.9	0.0314
Proposed (p -norm, $p=4$) + ChECDF + ECDF	0.800732	0.708	0.815	0.923	0.646	1.000	0.115828	40.1	0.0314
Proposed (mean) + ChECDF + ECDF	0.789645	0.708	0.770	0.923	0.646	1.000	0.115828	40.1	0.0314
Proposed (softmax, $\alpha=6$) + ChECDF + ECDF	0.758518	0.625	0.848	0.639	1.000	1.000	0.387571	6.2	0.0314
COPOD + ECDF	0.657870	0.583	0.826	0.808	0.595	0.261	0.225385	51.8	14.5946
IsolationForest + ECDF	0.650034	0.625	0.592	0.937	0.449	0.510	0.102496	106.9	5.2482
MatrixProfile ($m=64$) + ECDF	0.641433	0.458	0.576	0.900	0.912	0.658	0.137933	10.1	2.6198
Proposed (top2_mean) + ChECDF + ECDF	0.496494	0.167	0.848	0.620	0.022	1.000	0.405903	878.7	0.0314
Proposed (max) + ChECDF + ECDF	0.465328	0.167	0.723	0.631	0.000	1.000	0.395290	978.7	0.0314
Proposed (noisy_or) + ChECDF + ECDF	0.465328	0.167	0.723	0.631	0.000	1.000	0.395290	978.7	0.0314
ETS-residual + ECDF	0.310380	0.125	0.924	0.093	0.170	0.000	0.911057	424.7	39.7434

$CTS = \frac{1}{5}(CTS_C + CTS_S + CTS_H + CTS_D + CTS_R)$ with CTS_H based on $1-HHI$, and CTS_D , CTS_R based on inverted (smaller-is-better) scaling.

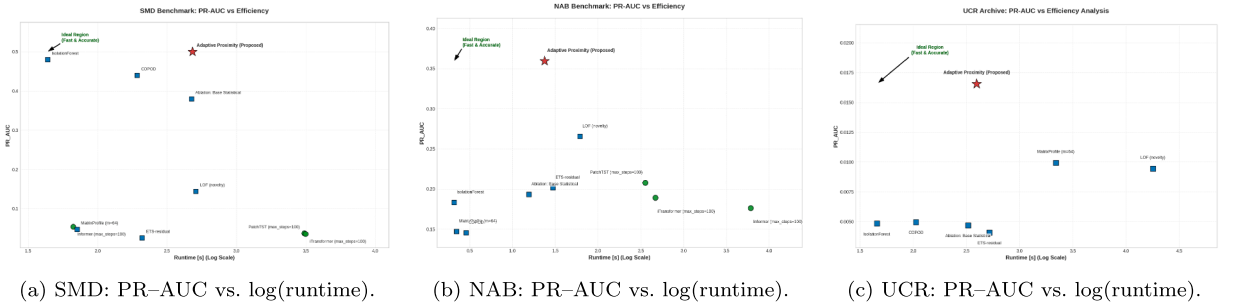


Fig. 1. Pareto analysis of accuracy vs. runtime using PR-AUC (vertical) and log(runtime) in seconds (horizontal). Each point corresponds to a method reported in Tables 3–5. Log scaling reflects multi-order-of-magnitude runtime differences. Pareto-dominated methods are those for which another method achieves higher PR-AUC at lower runtime.

On UCR at scale, as shown in Fig. 1 and Table 3, forecasting-based approaches are not competitive in either runtime or PR ranking, whereas the proposed method provides the strongest PR-AUC among evaluated baselines with tractable runtime, confirming that the approach remains viable under large-scale, heterogeneous streams.

PR@1% vs. runtime for operational alerting: While PR-AUC captures ranking quality, practical deployments often operate under explicit alert budgets. We therefore optionally repeat the Pareto analysis with PR@1% on the vertical axis (and runtime on the horizontal axis), aligning the evaluation with fixed-budget alerting policies reported in Tables 3–5. This view complements PR-AUC by highlighting operating-point behavior: on SMD, IsolationForest is advantageous at the strict 1% budget, whereas the proposed method preserves strong budget performance while providing substantially improved global PR ranking. On NAB, the proposed method yields very high PR@1% with moderate runtime, supporting conservative alerting when false alarms are costly. On UCR, where absolute PR values are small, PR@1% emphasizes the practical advantage of the proposed approach over slower shape-based baselines, even when ROC-AUC may favor alternative methods. We stress that these operating-point comparisons are policy-dependent; consequently, Section 4.3 further reports budget-sweep curves to avoid over-interpreting any single threshold.

Overall, the Pareto analysis provides an interpretable and deployment-relevant summary of the main results: the proposed framework consistently delivers strong PR-based accuracy with runtime characteristics that are compatible with real-time or near-real-time monitoring, while classical baselines remain useful as fast alternatives at specific budget regimes. This reinforces the manuscript’s central claim that proximity-based adaptive scoring with robust stabilization and leakage-safe calibration targets a practically favorable accuracy–compute trade-off; see Fig. 2.

4.3. Budget sensitivity

We report the main-text budget-sweep analysis on NAB and SMD because, together, they bracket the two deployment regimes targeted by our study and therefore make the operational point with minimal redundancy. NAB represents *rare-event streaming telemetry* with tight alert budgets where false alarms dominate operational cost; SMD represents *heterogeneous multi-sensor server telemetry* where anomaly mechanisms vary across series and the practical question is how quickly recall can be expanded as review capacity increases. In both cases, the same leak-free threshold-transfer protocol is used— $\tau(q)$ is learned exclusively on `calib_fit` via a deterministic tie-safe top- k rule and applied unchanged to `calib_eval`—so the resulting Precision@ q /Recall@ q curves directly quantify the precision–recall trade-off under fixed alarm-rate constraints.

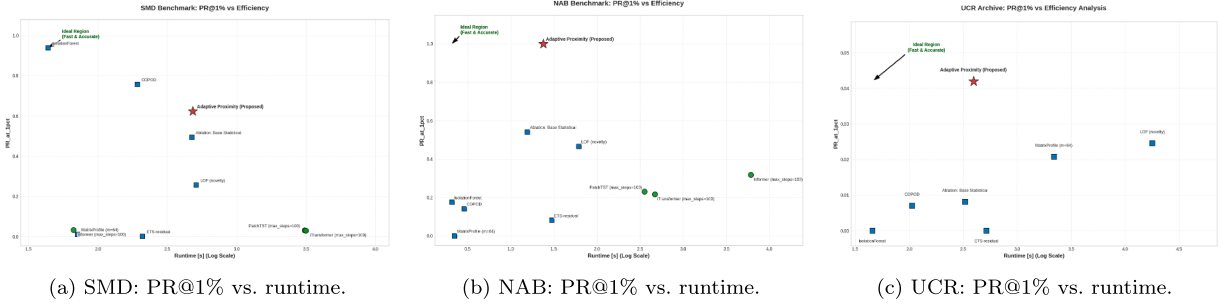


Fig. 2. Optional Pareto analysis under a fixed 1% alert budget, using PR@1% (vertical) and runtime in seconds (horizontal). This view emphasizes policy-driven operating-point performance and complements the PR-AUC analysis in Fig. 1.

We deliberately do not include a third full set of budget curves for UCR in the main text. UCR follows the same threshold-transfer protocol and yields the same type of operational conclusion, but adds substantial visual and narrative repetition while increasing figure count without expanding the coverage of deployment conditions beyond what is already established by NAB and SMD. Accordingly, the main text focuses on the two benchmarks that most clearly expose the operational extremes of interest, keeping the presentation compact and non-redundant.

4.3.1. Budget sensitivity on NAB (Operational trade-off)

We analyze how performance varies under an explicit alert budget q (fraction of points flagged). For each method, the decision threshold $\tau(q)$ is learned *exclusively* on `calib_fit` via a deterministic, tie-safe top- k rule and then applied *unchanged* to `calib_eval`. This strictly leak-free protocol yields Precision@ q and Recall@ q curves that reflect deployment-time budget constraints.

Importantly, our method is most advantageous in the operationally relevant low-budget regime. For $q \in \{0.1\%, 0.2\%, 0.5\%, 1\%\}$, it maintains near-perfect precision (i.e., essentially no false alarms at the smallest budgets), while Recall@ q increases monotonically as the budget is relaxed. This indicates that additional alerts primarily recover missed anomalies rather than introducing spurious detections. Consistent with this behavior, the Δ Precision@ q curves remain positive against both the statistical-only ablation and LOF across the explored budgets, showing that, when thresholds are learned on `calib_fit` and transferred to `calib_eval`, proximity-aware scoring translates directly into fewer false positives per unit budget and a more favorable precision-recall trade-off under realistic alerting constraints, as shown in Fig. 3.

4.3.2. Budget sensitivity on SMD (Operational trade-off)

We analyze performance under an explicit alert budget q , defined as the fraction of time points flagged as anomalous. For each method, the operating threshold $\tau(q)$ is *learned exclusively* on `calib_fit` using a deterministic, tie-safe top- k rule ($k = \lceil q|\text{calib_fit}| \rceil$), and then *applied unchanged* on `calib_eval`. This yields leak-free Precision@ q and Recall@ q curves that match deployment constraints where the alarm rate is fixed in advance.

Fig. 4 shows that at extremely tight budgets ($q \leq 0.5\%$), heavy-tail outlier detectors (e.g., IsolationForest/COPOD) can achieve very high Precision@ q by selecting only the most extreme points, albeit with limited recall. In contrast, the proposed proximity-fused scoring becomes competitive from moderate operational budgets onward. At $q \approx 1\%$, OUR reaches a substantially higher operating-point balance, and for $q \in [2\%, 5\%]$ it provides the strongest recall while maintaining practical precision, indicating that additional alarm capacity is used to surface diverse anomaly regimes rather than repeatedly concentrating on a narrow subset of points.

To isolate operational gains, Fig. 5 reports Δ Precision@ q and Δ Recall@ q versus two relevant baselines: the statistical-only ablation (ABL) and LOF. For $q \geq 1\%$, OUR yields consistently positive Δ Recall@ q and positive Δ Precision@ q against LOF, with the margin widening as the budget increases. Against ABL, OUR provides a stable recall improvement across q , while precision gains are more modest and most pronounced in the mid-budget regime. Overall, the sweep indicates that the proposed method is most effective in realistic monitoring settings where operators can allocate $q \approx 1\% - 5\%$ of points for review, delivering a favorable precision-recall trade-off under a strictly leak-free thresholding protocol.

4.4. Unlabeled intel: Triage results

Intel Berkeley telemetry is unlabeled; therefore, we evaluate deployment triage utility using the leak-free protocol and Composite Triage Score (CTS) defined in Section 2.7.

CTS-ranked results: Table 6 reports the CTS ranking together with the normalized CTS components and the key raw quantities (HHI, mean event duration, runtime) computed on `calib_eval` at $q = 1\%$. The highest CTS is achieved by Proposed(ProxFuse=invMAD_weighted_on_cf) + ChECDF(cf) + ECDF(cf), indicating the most favorable overall triage profile under the chosen operating policy: broad fleet coverage, adequate cross-split stability after calibration transfer, low concentration of alerts across nodes, and non-degenerate event durations, all at negligible runtime.

Operational trade-offs and failure modes: On unlabeled telemetry, a high score alone is insufficient; the triage objective penalizes *operationally degenerate* behaviors. Several fusion rules can be computationally inexpensive yet yield weak screening utility by

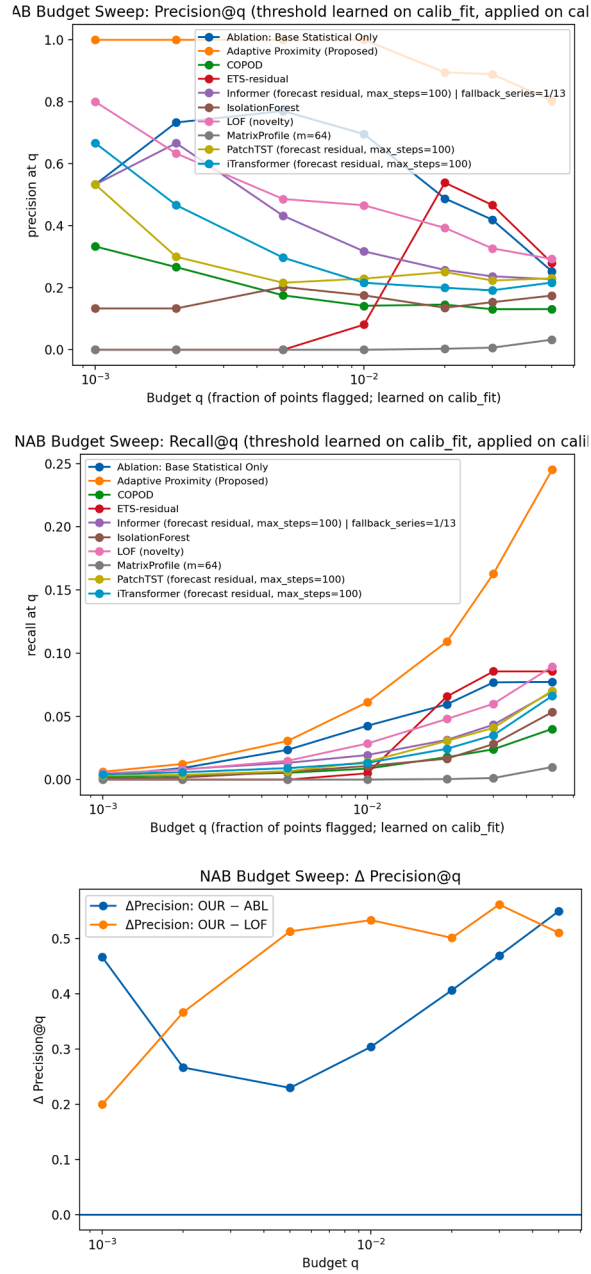


Fig. 3. NAB budget sweep on calib_eval: Precision@ q (top), Recall@ q (middle), and Precision gain Δ Precision@ q relative to key baselines (bottom). Thresholds are learned on calib_fit and applied leak-free to calib_eval.

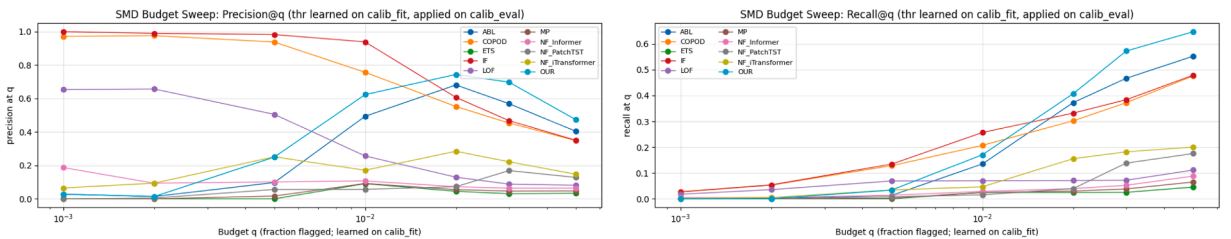


Fig. 4. SMD budget sweep on calib_eval: Precision@ q (left) and Recall@ q (right), with $\tau(q)$ learned on calib_fit and applied unchanged on calib_eval.

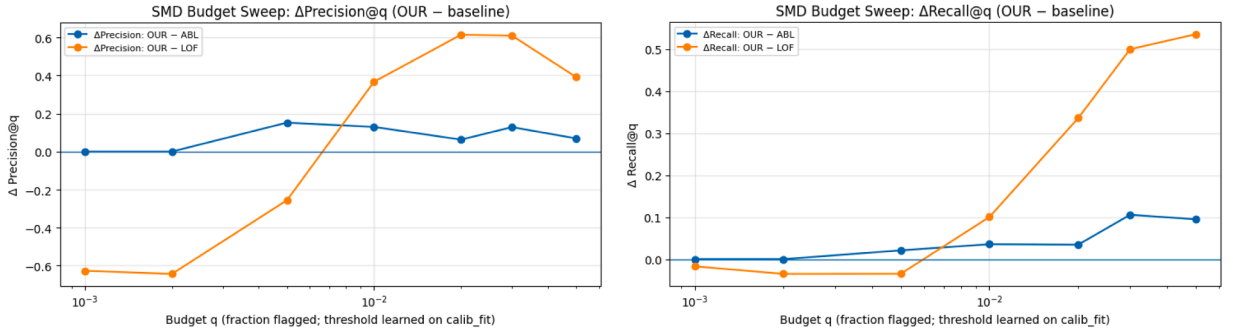


Fig. 5. SMD operational deltas: Δ Precision@ q and Δ Recall@ q for OUR relative to ABL and LOF. Positive values indicate an advantage for OUR at the same alert budget q .

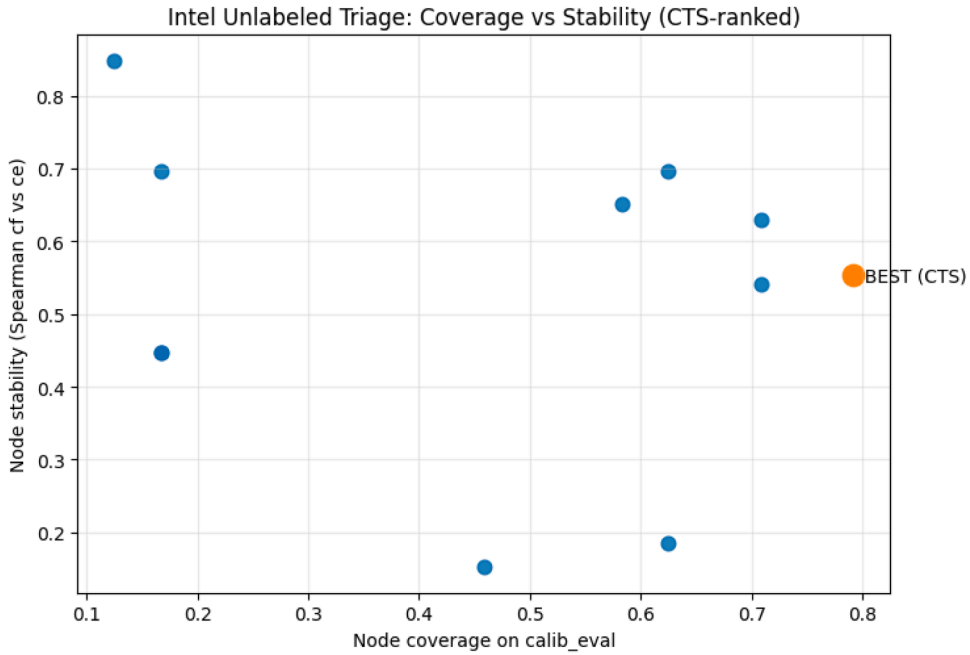


Fig. 6. Intel unlabeled triage under a leak-free $q = 1\%$ alert budget. Each point is a method positioned by node coverage on calib_eval and node-wise stability (Spearman agreement of mean scores between calib_fit and calib_eval). The CTS-best method is highlighted.

(i) concentrating alerts on a small subset of nodes (high HHI), or (ii) producing extremely persistent alert segments (inflated mean duration), which collapses operator attention into long low-granularity intervals. Conversely, methods that appear stable in a rank sense across calib_fit and calib_eval can still be unattractive if coverage is low or runtime is prohibitive. CTS makes these tensions explicit by combining the five criteria into a single, leak-free ranking aligned with deployment screening rather than accuracy.

Coverage–stability view: Fig. 6 provides a compact view of the dominant trade-off between node coverage on calib_eval and cross-split stability (Spearman agreement of node-wise mean scores between calib_fit and calib_eval), with the CTS-selected operating point highlighted.

4.5. Explainability (SHAP)

The proximity core is unsupervised and does not expose a native parametric mapping from input features to scores. Therefore, for interpretability we train a lightweight supervised *surrogate head* (LightGBM) on calib_fit to predict the calibrated anomaly probability, and we report SHAP attributions of this head. This analysis is optional and does not alter the core detector or any reported detection metric.

SHAP for NAB:

To ensure that the proposed detector remains transparent at deployment time, we analyze its decision logic using SHapley Additive exPlanations (SHAP). Importantly, all explanations reported for NAB are computed on the *leak-free* evaluation block (calib_eval) produced by the external 3-way temporal splitter; the model is trained without using calib_eval labels, and SHAP is computed with

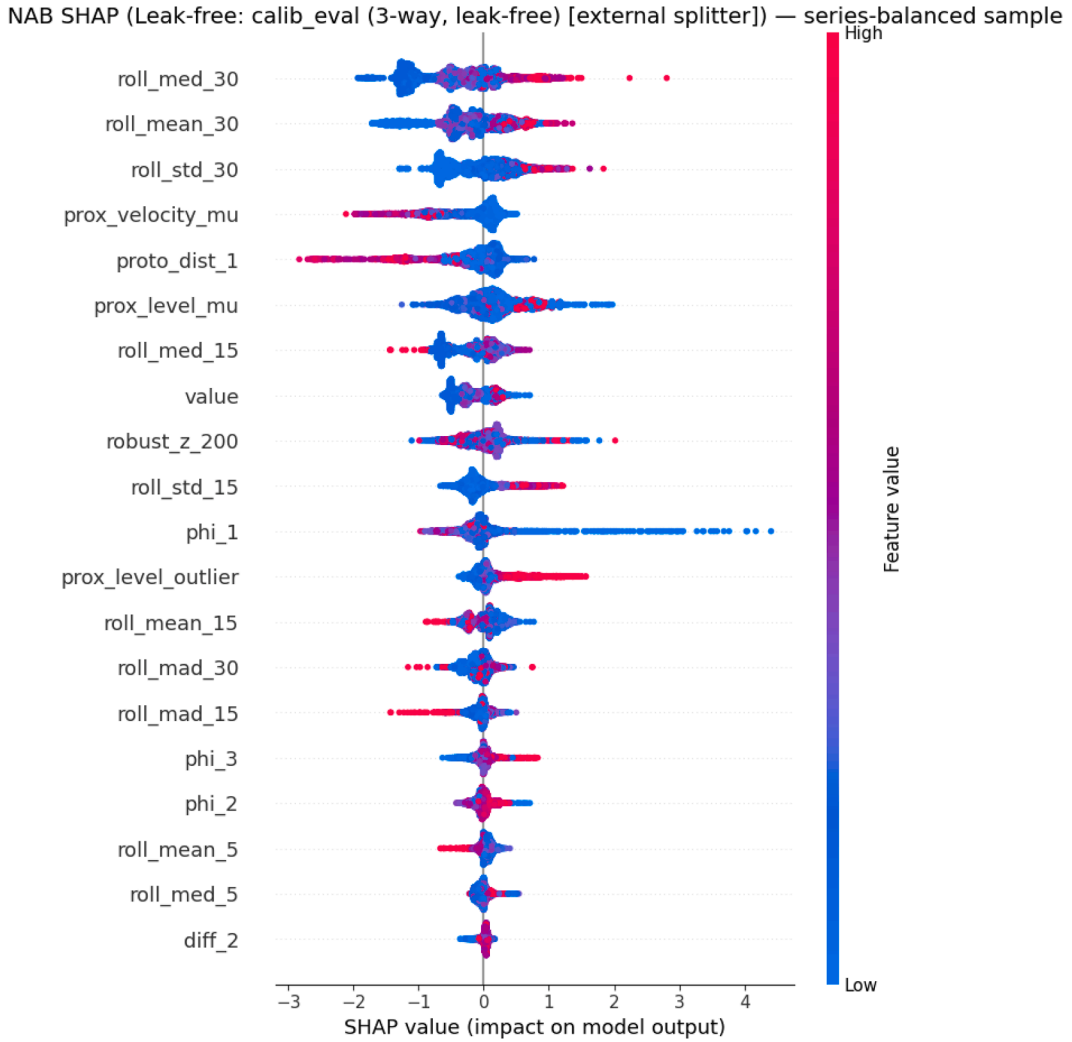


Fig. 7. NAB SHAP summary (leak-free calib_eval, external 3-way splitter; series-balanced sample). SHAP values are computed for the *calibrated anomaly probability*. Each point corresponds to an evaluation instance; color encodes the feature value (low to high), and the horizontal axis shows the signed contribution to the model output.

respect to the model output *calibrated probability* (i.e., the post-calibration anomaly probability used for ranking and thresholding). For NAB, the SHAP evaluation set contains 14,744 points from 13 series with 3141 positives, and we use a series-balanced sampling scheme (169 points per series; 5,132 total) so that long streams do not dominate the global explanation.

Fig. 7 summarizes global attribution patterns via a SHAP beeswarm plot. The ranking is driven primarily by robust statistical descriptors computed over medium windows, led by `roll_med_30`, `roll_mean_30`, and `roll_std_30`. Proximity-driven descriptors then appear prominently, with `prox_velocity_mu`, `proto_dist_1`, and `prox_level_mu` among the top contributors. These results are consistent with the intended design: statistical features encode local level and dispersion shifts, while proximity features quantify deviation relative to learned neighborhood structure, thereby supporting detection under drift and heterogeneous regimes.

Table 7 reports the top-10 features by mean absolute SHAP magnitude. To provide a higher-level view, we also aggregate mean absolute attributions by feature family (Statistical, Proximity, Latent, Raw). The resulting group shares indicate that Statistical features account for 61.0% of the total mean|SHAP|, Proximity features account for 25.2%, Latent features contribute 8.0%, and the Raw signal value contributes 5.8%. This decomposition highlights that the model relies on a blend of classical time-series statistics and neighborhood-aware proximity cues, with latent components providing a smaller but non-negligible supporting signal, as summarized in **Table 8**.

Beyond aggregate importance, we assess whether the global explanation is representative across heterogeneous NAB streams. For each of the 13 series, we compute a per-series feature-importance ranking and compare it to the global ranking using Spearman correlation. The mean (median) rank correlation is 0.867 (0.870), indicating that the explanation is broadly stable across series rather than being driven by a small subset of streams. At the same time, proximity-related features consistently appear among the top

Table 7

NAB (leak-free calib_eval) top-10 features by mean absolute SHAP value (series-balanced sample).

Feature	mean SHAP	Group
roll_med_30	0.612431	Statistical
roll_mean_30	0.423493	Statistical
roll_std_30	0.388476	Statistical
prox_velocity_mu	0.352357	Proximity
proto_dist_1	0.324283	Proximity
prox_level_mu	0.323765	Proximity
roll_med_15	0.307884	Statistical
value	0.278519	Raw
robust_z_200	0.225833	Statistical
roll_std_15	0.225606	Statistical

Table 8

NAB SHAP group contributions (sum of mean|SHAP|) and shares.

Group	Sum mean SHAP	Share (%)
Statistical	2.906821	61.0%
Proximity	1.201264	25.2%
Latent	0.382476	8.0%
Raw	0.278519	5.8%

contributors for most series (typically 2–4 proximity features within the per-series top-10), reflecting that neighborhood deviation signals are repeatedly useful across diverse regimes (e.g., traffic, operational metrics, and known-cause failures).

Global importance explains *which* signals the model uses on average, whereas local explanations clarify *why* a specific timestamp receives a high calibrated anomaly probability. Since SHAP values are computed directly on calib_eval for the calibrated output, local attributions can be examined at any evaluation point (including around known NAB anomaly intervals) to separate (i) level/dispersion-driven evidence captured by rolling statistics from (ii) neighborhood-deviation evidence captured by proximity descriptors. This calibrated-probability view supports operational interpretability: the same explanation space that drives ranking and budget-based decision rules also drives the per-point narratives used during investigation.

SHAP for SMD:

We next examine the interpretability of the proposed detector on the Server Machine Dataset (SMD) using SHapley Additive exPlanations (SHAP). In contrast to single-stream benchmarks, SMD contains multiple heterogeneous servers with distinct operating regimes. To obtain a faithful global explanation while respecting this heterogeneity, we adopt the following strategy: for each of the 28 series, we re-train the model on that series (leak-free, series-wise), compute SHAP values on an evaluation subset, sample 800 evaluation points per series, and then aggregate SHAP values across series to form a global summary. This aggregation prevents any single long or high-variance stream from dominating the explanation.

Fig. 8 reports the aggregated SHAP beeswarm plot across 28 series. The dominant contributors are rolling statistical descriptors computed per channel (e.g., roll_mean_30_x05, roll_mean_30_x06, roll_mean_30_x15), indicating that medium-horizon shifts in local level remain the primary drivers of anomaly probability in SMD. Proximity-aware signals also appear among the top contributors, notably proto_dist_1 and prox_level_mu, consistent with the role of neighborhood deviation in capturing departures that are not fully explained by univariate statistics. An additional high-ranking input is x04 (grouped as *Other*), reflecting that certain raw channels can carry direct discriminative signal depending on the server and metric (Table 9).

The top-10 global features by mean absolute SHAP magnitude are:

x04 (0.1004),
 roll_mean_30_x05 (0.0772),
 roll_mean_30_x06 (0.0766),
 roll_mean_30_x15 (0.0754),
 proto_dist_1 (0.0736),
 roll_mean_5_x00 (0.0627),
 roll_std_30_x13 (0.0582),
 roll_mean_15_x06 (0.0520),
 roll_std_30_x34 (0.0515),
 and prox_level_mu (0.0515).

At the feature-family level, Statistical descriptors account for 76.38% of the total mean|SHAP|, followed by Other (12.73%), Proximity (7.17%), and Latent (3.71%). This profile indicates that SMD anomalies are predominantly expressed as sustained distributional shifts across channels, while proximity and latent components provide complementary evidence for cases where relative neighborhood structure or more compressed representations are informative.

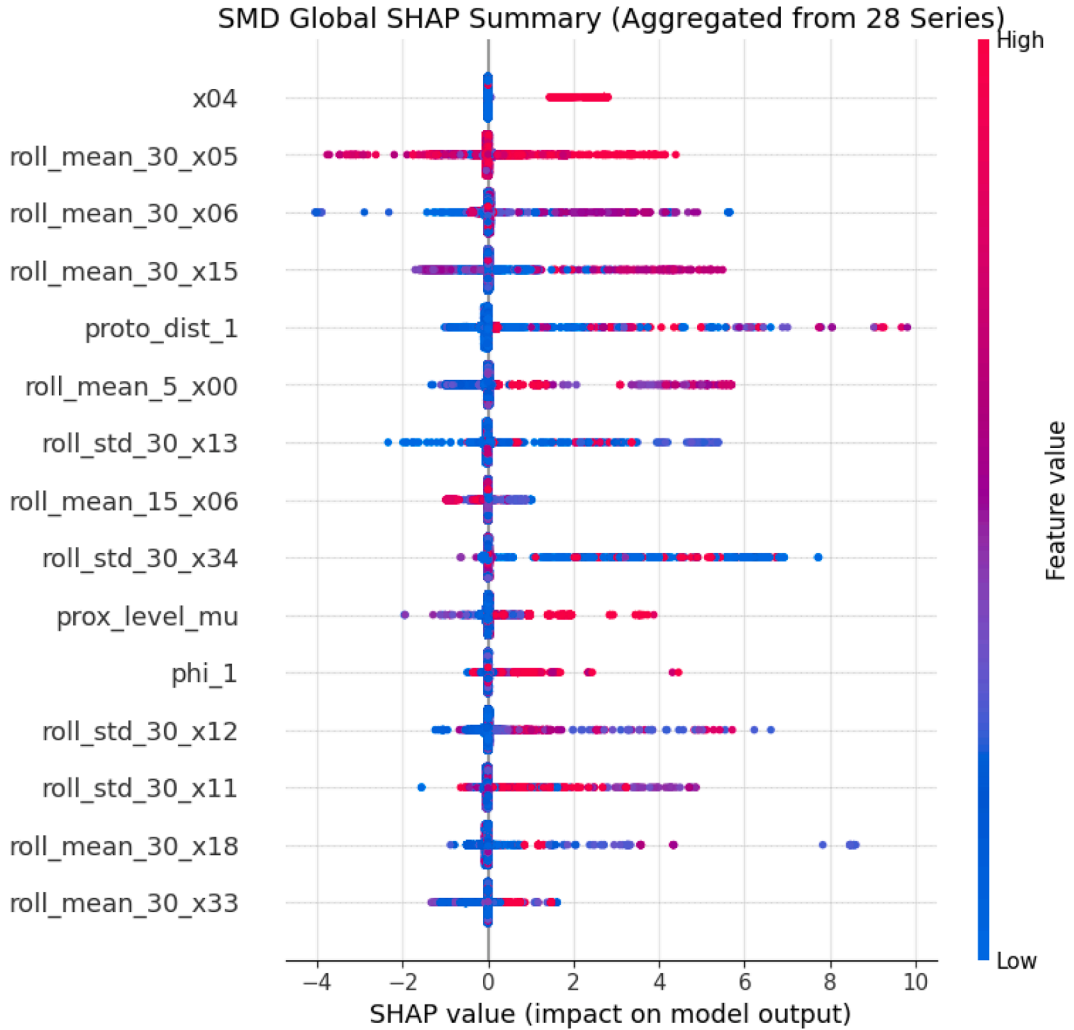


Fig. 8. SMD global SHAP summary aggregated from 28 series. SHAP values are aggregated from series-wise models using 800 evaluation samples per series. Each point corresponds to an evaluated timestamp; color encodes feature magnitude (low to high), and the horizontal axis indicates signed contribution to the model output.

To quantify whether the global explanation is consistent across servers, we compute series-level importance rankings and compare them against the aggregated ranking. The mean Spearman rank correlation is 0.5891, and the average top-5 overlap is 0.69/5. This indicates moderate stability: a shared set of statistical drivers recurs across servers, while the exact channel-level ordering varies with server-specific operating conditions and sensor semantics.

In practical terms, SMD benefits from a global interpretability narrative (rolling level/dispersion shifts dominate), while root-cause analysis at the server level should still rely on local attributions to identify which channels and mechanisms are responsible in a given stream, with the aggregate feature-family contributions summarized in [Table 10](#).

4.5.1. SHAP for UCR (Calibrated probability; leak-free calib_eval)

To interpret the proposed detector beyond aggregate accuracy, we compute SHAP values on the *deployment-like* evaluation block (`calib_eval`) obtained via a per-series, time-ordered three-way split (train / `calib_fit` / `calib_eval`). Importantly, SHAP is computed on the model output expressed as *calibrated probability* (i.e., the post-calibration anomaly probability), so feature attributions directly explain the final decision score used in thresholding and budget-based selection.

For UCR, the leak-free evaluation pool contains 2,386,845 points across 250 series, with 10,018 positives. To prevent long series from dominating the explanation, we construct a series-balanced SHAP sample with a target of ~80 points per series, capped at 20,000 total points (resulting in $n = 20,000$ with 100 positives). [Fig. 9](#) summarizes global feature effects.

Across the global ranking, statistical descriptors dominate the explanation mass, accounting for 76.6% of the summed mean absolute SHAP, while proximity features contribute a substantial 20.5% and raw signal terms explain the remaining 2.9%. The top contributors

Table 9
SMD top-10 global features by mean absolute SHAP value (aggregated from 28 series).

Feature	mean SHAP	Group
x04	0.100395	Other
roll_mean_30_x05	0.077171	Statistical
roll_mean_30_x06	0.076557	Statistical
roll_mean_30_x15	0.075419	Statistical
proto_dist_1	0.073640	Proximity
roll_mean_5_x00	0.062703	Statistical
roll_std_30_x13	0.058228	Statistical
roll_mean_15_x06	0.052011	Statistical
roll_std_30_x34	0.051520	Statistical
prox_level_mu	0.051466	Proximity

Table 10
SMD SHAP group contributions (sum of mean|SHAP|) and shares.

Group	Sum mean SHAP	Share
Statistical	1.630793	76.38%
Other	0.271878	12.73%
Proximity	0.153117	7.17%
Latent	0.079220	3.71%

Table 11
UCR SHAP group contributions (sum of mean|SHAP|) and percentage shares computed on the leak-free calib_eval block (3-way split; local splitter).

Group	Sum mean SHAP	Share (%)
Statistical	3.871848	76.6
Proximity	1.036067	20.5
Raw	0.146547	2.9

by mean(|SHAP|) are roll_med_15 (0.7098), roll_med_30 (0.5283), prox_level_mu (0.4544), roll_mean_30 (0.3982), roll_mad_30 (0.3676), proto_dist_1 (0.3496), roll_std_30 (0.3272), robust_z_200 (0.3260), pct_change (0.2484), and roll_mean_5 (0.1646). Collectively, this profile indicates that the model primarily responds to robust local level and scale changes (rolling median/mean, MAD/STD, robust z-scores), while the proximity block (prox_level_mu, proto_dist_1, and related terms) provides additional geometric evidence that systematically shifts calibrated anomaly probability beyond what classical statistics alone capture.

The UCR explanation is also stable across series: the mean Spearman rank correlation between series-wise feature ranks and the global rank is 0.822 (median 0.829; $n = 250$ series). This indicates that, despite heterogeneity in signal morphology, the model relies on a consistent set of mechanisms across the benchmark. In particular, proximity descriptors repeatedly appear among the top features in many series, supporting the intended role of proximity-aware modeling as a transferable complement to robust statistical change descriptors under leak-free, time-forward evaluation (Table 11).

4.5.2. Intel (Unlabeled): Distillation-SHAP for interpreting the proximity score

The Intel Berkeley Lab corpus is largely unlabeled, which prevents label-driven local case studies (e.g., explaining why a specific timestamp is truly anomalous). Instead, we focus on explaining the *internal proximity score* that drives our detector in deployment. Concretely, we apply a distillation-based SHAP analysis: we train a lightweight surrogate model (LightGBM regressor) to mimic the learned proximity score prox_level_mu using *only* raw and statistical inputs, and then compute SHAP on a leak-free evaluation split.

Why proximity features do not appear in Intel SHAP: In this block, prox_level_mu is the *distillation target*. To avoid circular explanations, we explicitly exclude prox_*, proto_*, phi_*, and knn_* from the surrogate inputs (anti-circular feature filtering). Therefore, SHAP is computed over raw/statistical variables that *drive* the proximity score rather than the proximity features themselves. This design ensures that the attributions answer the intended question: *which measurable sensor patterns (raw and rolling statistics) make the proximity score increase or decrease?*

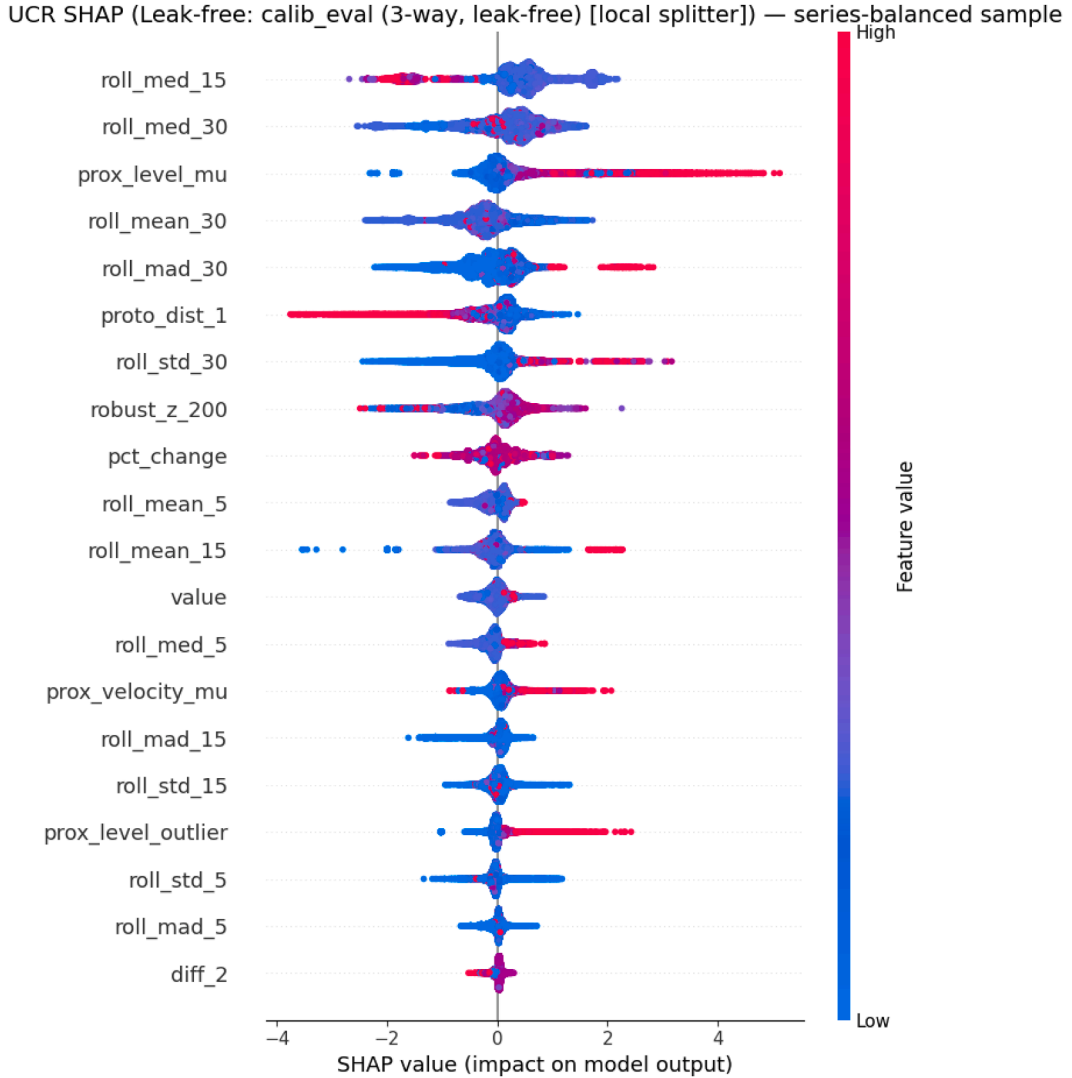


Fig. 9. UCR global SHAP summary on the leak-free calib_eval block (3-way split; local splitter), using a series-balanced sample. SHAP values explain the *calibrated* anomaly probability: points to the right (left) increase (decrease) the predicted anomaly probability, and color encodes the feature value (blue: low, red: high).

Leak-free protocol and SHAP sampling: We enforce time causality by splitting each sensor node (24 nodes) in chronological order into a 70% training prefix and a 30% calibration block, which is further split into calib_fit/calib_eval (50/50). All fitted preprocessing (imputation statistics, scaling, PCA) is learned on the training prefix only; any imputation on held-out blocks uses *train-only* medians. SHAP is computed post hoc on a node-balanced sample drawn from calib_eval (total $n = 1488$, ≈ 62 points per node), preventing long node traces from dominating global importance. SHAP is used for interpretability only and does not influence any fitting or operating-point selection.

Global drivers of prox_level_mu: Fig. 10 reports the global SHAP summary for the distilled prox_level_mu model under the leak-free evaluation protocol. The dominant contributors are voltage-related robust deviations and residual terms, followed by illumination and humidity/temperature rolling statistics. This pattern is consistent with the intended semantics of prox_level_mu: the proximity score rises when multiple sensor channels exhibit structured departures from their recent baseline (robust z-scores, residuals, and short-horizon variability), rather than reacting to raw levels in isolation. Since the dataset is unlabeled, we treat these results as an interpretability audit of the scoring mechanism, not as a claim about ground-truth anomaly causation.

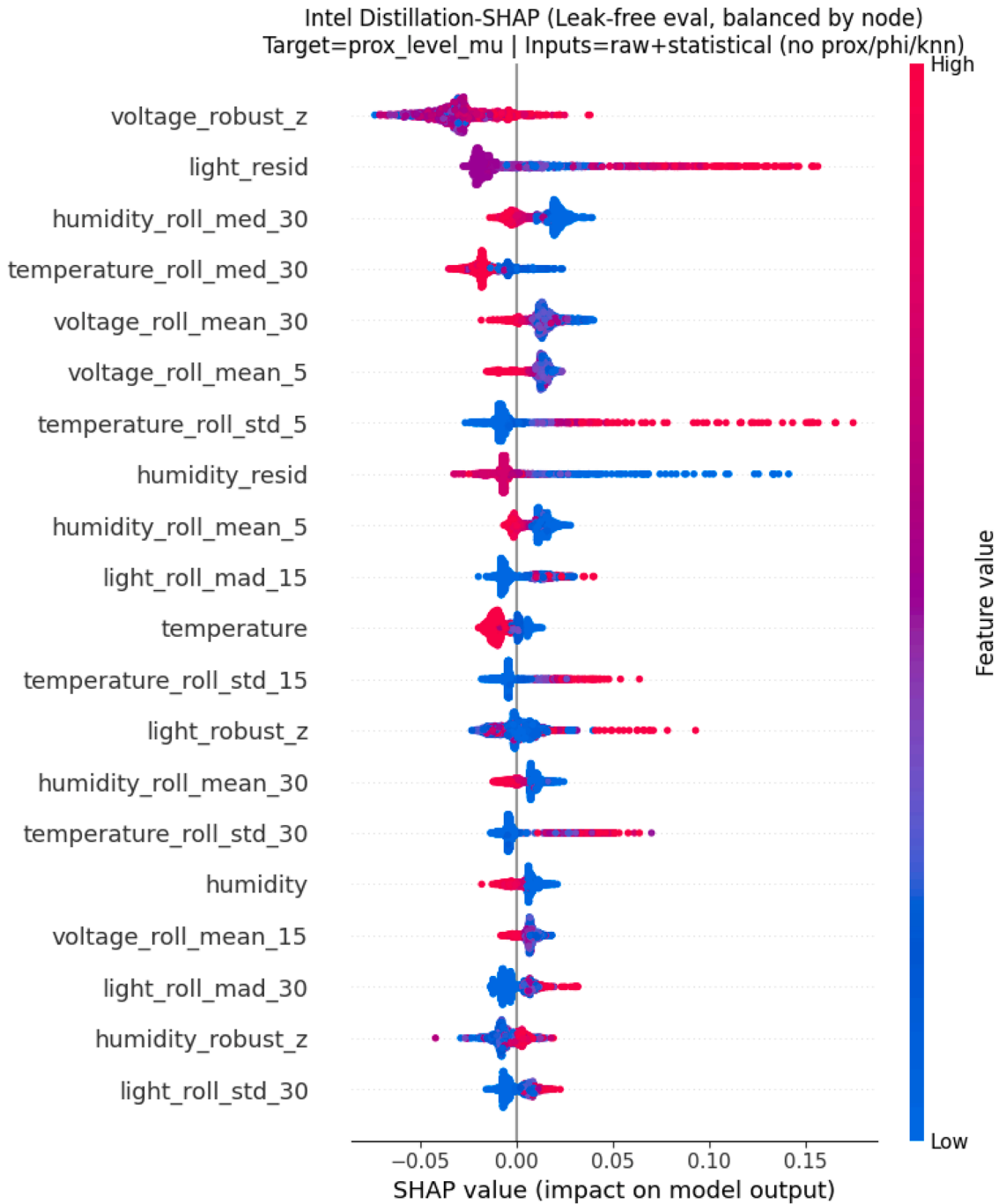


Fig. 10. Intel Distillation-SHAP summary on the leak-free evaluation block, balanced by node. The surrogate model predicts `prox_level_mu` using only raw and statistical features (no `prox/phi/knn/proto`) to avoid circular attributions.

5. Discussion

5.1. What the results mean

The results should be interpreted through the manuscript's deployment objective: reliable ranking and fixed-budget alerting under drift, extreme imbalance, and constrained compute, evaluated under a strictly leakage-safe regime (train / `calib_fit` / `calib_eval`). Under this objective, three consistent patterns emerge.

PR-oriented ranking strength on NAB and SMD: On NAB, the proposed framework attains the strongest pooled PR-AUC (0.359) under pointwise evaluation on `calib_eval` (Table 4), and it yields a conservative but operationally clean alerting regime at a 1% budget (PR@1%= 1.000, R@1%= 0.061). On SMD, the proposed method achieves the strongest PR-AUC (0.500) (Table 3), indicating improved global ordering of positives in a multivariate, heterogeneous telemetry setting. These outcomes align with the design choice to fuse multiple train-referenced proximity cues under rolling robust stabilization: the detector is optimized to produce stable rankings under drift and heavy-tailed noise, where PR-based summaries are more representative of operational utility than ROC-based summaries [32,33].

Trade-offs at fixed budgets and on UCR scale: The evaluation explicitly separates ranking metrics (ROC-AUC/PR-AUC) from policy-driven fixed-budget behavior (PR@1%, R@1%). This separation matters in practice: a method can achieve a favorable PR-AUC while being less competitive at a specific tight budget, and the reverse can hold for heavy-tail detectors that concentrate alarms on extreme points. On SMD, IsolationForest attains higher PR@1% and R@1% at the 1% budget, while our method attains higher PR-AUC. This is a genuine operating trade-off rather than an inconsistency: it reflects different scoring geometries and how they interact with an alarm-rate constraint. On UCR, where the benchmark is intentionally hostile (heterogeneous series, extreme sparsity, $\approx 15.9\text{M}$ points), absolute PR-AUC values are small across methods (Table 5). In this regime, shape-based scoring (Matrix Profile) improves ROC-AUC, while the proposed method improves budgeted alerting quality (PR@1% and R@1%) with tractable runtime. The appropriate conclusion is therefore deployment-oriented: the proposed framework provides a favorable Pareto point for budgeted screening at scale, rather than uniform dominance on all global metrics.

Intel is a triage validation, not an accuracy claim: Intel Berkeley Lab telemetry is unlabeled. We therefore avoid any supervised detection claims on Intel and instead evaluate deployment triage utility under a fixed alert budget using leak-free threshold transfer and the Composite Triage Score (CTS) defined in Section 2.7. The highest CTS is achieved by the proposed `invMAD`-weighted `ProxFuse` variant with calibration-transfer components (Table 6), reflecting a balanced operational profile across coverage, cross-split stability, concentration, event duration, and runtime. This evidence supports the intended role of Intel in the study: demonstrating that the calibrated, budgeted scoring mechanism yields non-degenerate and operationally plausible alert patterns under realistic missingness and long-duration streams, without overstating ground-truth detection performance.

5.2. Transformer baselines: Why compute budget matters

Transformer forecaster residual baselines (Informer, iTransformer, PatchTST) are included as contemporary references where computationally feasible, evaluated under an explicit compute policy (`max_steps` = 100 with fail-soft fallback) and the same leak-free calibration/threshold protocol. Two observations motivate this design choice.

First, the wall-clock spread is multi-order-of-magnitude. Even with bounded steps, forecaster residual pipelines can be substantially slower than proximity-based scoring, and they can become vulnerable to long-job interruption in constrained environments. This is visible on NAB and SMD, where transformer residual runtimes are orders of magnitude larger than the proposed approach (Tables 3 and 4). Second, compute budgets affect the realized model quality of forecasters; a bounded training schedule is an operational reality for many deployments and should be reflected in experimental comparisons. Accordingly, we interpret transformer baselines as compute-bounded references rather than unconstrained state-of-the-art training runs. Under this interpretation, the results support a practical conclusion: the proposed method can match or exceed PR-oriented ranking quality while maintaining a predictable runtime profile that is compatible with continuous monitoring.

5.3. Deployment guidance

The framework is designed to be actionable under limited supervision and fixed review capacity. The following guidance summarizes how the reported protocol translates to deployment.

Selecting an alarm budget: The alert budget q encodes operational review capacity (fraction of points flagged per stream). In this study, we use $q = 1\%$ as a representative tight-budget regime and report budget sweeps on NAB and SMD (Section 5.3). In deployment, q should be chosen via a capacity-driven budget sweep: start with a conservative range (e.g., $q \in [0.1\%, 1\%]$) when false alarms are costly, then expand toward moderate budgets (e.g., $q \in [1\%, 5\%]$) when the goal is broader coverage and higher recall. The tie-safe exact top- k rule used here is recommended when scores are quantized or when ties are frequent, since it prevents unstable thresholding.

Calibration when labels exist: On labeled benchmarks, isotonic regression is used as a monotone calibrator trained strictly on `calib_fit` and applied unchanged to `calib_eval`. This separation is essential: fitting calibration or choosing an operating point on the same block used for reporting inflates performance. In operational settings where labels arrive with delay, calibration should be updated only on past labeled segments and validated on a held-out block that simulates future data. If the calibration-fit block contains a single class (rare anomalies), the identity fallback is a safe option that preserves leak-freedom and avoids ill-posed fits.

Unlabeled fleets and triage: When labels are unavailable (Intel-like settings), *label-free score normalization* (e.g., ECDF/ChECDF) can still be used for screening and prioritization under a fixed budget, combined with stability and concentration checks. The CTS components provide a practical audit: broad coverage, stable node ranking across time blocks, low alert concentration, non-degenerate event durations, and bounded runtime. This operational lens helps distinguish methods that generate superficially high scores from those that yield actionable screening behavior.

Edge/streaming execution: The method is series-local and naturally streaming compatible: preprocessing and embedding are train-only, rolling-MAD scaling is causal, and scoring can be produced per timestamp. Peak memory is controlled by processing streams

independently (manifest/parts workflow). The dominant computational term is train-referenced neighborhood querying; deployments with strict latency constraints should consider approximate nearest-neighbor indexing built on the fixed training reference, which preserves the leak-free property as long as the index is constructed using past data only (Section 2.8).

5.4. Limitations and future work

The study has several limitations that point to concrete future directions.

Score construction and multiscale structure: Rolling-MAD stabilization is applied to a single embedding coordinate and the proximity cues operate at a fixed embedding dimension ($d = 3$). While this choice improves robustness and feasibility, it can miss multiscale anomaly signatures or regime-dependent temporal structure. Extensions could incorporate multiscale proximity (multiple window sizes and scale channels) and explicit change-point awareness to better separate abrupt transitions from persistent drift.

Scalability of train-referenced neighborhoods: The computational bottleneck is the train-referenced neighborhood queries. While series-local processing bounds memory, k NN queries dominate runtime for long streams. Future work includes approximate or incremental variants: approximate nearest-neighbor indices, prototype-based compression of the training reference, and controlled reference refresh schedules that remain strictly time-forward.

Benchmark coverage and evaluation semantics: Intel is unlabeled and is used for triage validation; supervised accuracy cannot be assessed there. On NAB, the main tables report pointwise pooled metrics under a budget policy, while NAB also supports event-window scoring. Expanding the evaluation to additional industrial multivariate datasets with dense labels, and reporting complementary event-based scoring views where appropriate, would strengthen coverage across annotation conventions and deployment regimes.

Calibration under distribution shift: Calibration is label-dependent and can degrade under major distribution shifts. The present protocol enforces leak-freedom and demonstrates stable behavior across the evaluation blocks, yet real deployments may require periodic recalibration on recent labeled data (when available), together with drift monitoring and reliability diagnostics (e.g., calibration curves over time blocks).

Threats to validity primarily include temporal leakage, tuning bias in operating-point selection, and runtime comparability across methods. We mitigate these risks via per-series chronological splits, train-only calibration and threshold learning (applied unchanged to `calib_eval`), and fixed compute policies with wall-clock end-to-end runtime reporting on identical hardware. Remaining differences reflect inherent method cost profiles and are interpreted through Pareto and budget-sweep analyses rather than single-metric claims.

6. Conclusion

We present a deployment-oriented anomaly detection framework for non-stationary time series that integrates: (i) a train-referenced adaptive score formed by fusing complementary normalized membership cues (level departure, a density cue with a safe fallback, velocity in embedding-increment space, and prototype distance), (ii) rolling-MAD stabilization for robustness under drift and heavy-tailed noise, and (iii) leak-free decision making with train-only fitting, isotonic calibration on `calib_fit`, and fixed-budget threshold transfer to `calib_eval`. Evaluation is strictly leakage-safe and series-wise, explicitly separating score formation, calibration and operating-point selection, and final reporting.

Across three labeled benchmarks (NAB, SMD, UCR) and one unlabeled deployment triage corpus (Intel Berkeley Lab), the method provides consistent evidence of practical utility. On NAB and SMD, it achieves the strongest PR-AUC under the leak-free protocol, while budgeted metrics expose meaningful operating trade-offs against classical baselines at tight budgets. On UCR at scale, it delivers a favorable budgeted screening point with tractable runtime, even when alternatives achieve higher ROC-AUC. On unlabeled Intel telemetry, we avoid supervised accuracy claims and instead demonstrate operational screening value under a fixed alert budget using CTS-based triage criteria, complemented by distillation-based interpretability audits of the proximity score.

Overall, the contribution is a cohesive, compute-feasible pipeline that produces deployable anomaly scores and calibrated probabilities under strict leak-free evaluation, supports fixed-budget operating policies, and provides aligned interpretability through SHAP-based analyses. The results motivate proximity-based adaptive scoring as a practical alternative to compute-heavy forecaster residual pipelines in continuous monitoring, and they identify clear future directions in multiscale proximity, change-point awareness, and scalable neighborhood approximations for large fleets and long-duration streams.

Data availability statement

The datasets used in this study are publicly available from their original sources: the Numenta Anomaly Benchmark (NAB) [5], the Server Machine Dataset (SMD) [40], the UCR Time Series Anomaly Archive [41,42], and the Intel Berkeley Research Lab sensor telemetry [7]. No new proprietary dataset is introduced.

To support reproducibility, we provide as supplementary materials the scripts to download and preprocess the data, construct per-series leak-free train/`calib_fit`/`calib_eval` splits, reproduce all tables/figures, and environment snapshots (including a full `pip freeze`), raw data are obtained from the original sources under their respective licenses.

CRedit authorship contribution statement

Ebubekir İnan: Writing – review & editing, Writing – original draft, Visualization, Validation, Software, Resources, Methodology, Formal analysis, Data curation, Conceptualization.

Data availability

Public datasets (NAB, Intel Berkeley Lab, UCR, KPI). We do not redistribute; repo/SI provide scripts to download originals plus leakage-safe splits, OOF calibration, and artifacts.

Declaration of competing interest

The author declares that he has no known competing financial interests or personal relationships that could have influenced the work reported in this paper.

Appendix A. Algorithmic summary of the leakage-safe adaptive proximity pipeline

A.1. Notation and split policy

For each series, samples are processed chronologically and split into contiguous blocks: train prefix X_{tr} (70%), calibration-fit X_{cf} (15%), and calibration-eval X_{ce} (15%). All fitting (causal feature engineering, imputation, scaling, PCA, proximity construction, supervised head training, probability calibration, and operating-point learning) is restricted to the appropriate *train-only* or *calib_fit-only* segment, and then applied unchanged to X_{ce} (strictly out-of-sample). We use a fixed alert budget q (default $q = 1\%$) learned on X_{cf} via deterministic exact top- k selection (argpartition-based in the implementation).

Algorithm 1 A1. Causal feature engineering and timestamp hygiene (per series).

Require: Raw series $\{(t_i, x_i)\}_{i=1}^n$ (timestamps and values), windows $\mathcal{W} = \{5, 15, 30\}$, robust-z window w_z (default 200).

Ensure: Feature matrix $\Phi \in \mathbb{R}^{n \times p}$ with *causal* rolling statistics.

- 1: **Sort** samples by timestamp t_i ascending (stable).
- 2: **Timestamp uniqueness (recommended):** if duplicate timestamps exist, make them unique by adding deterministic nanosecond offsets within each duplicate group (preserves chronological order).
- 3: **for all** $u \in \mathcal{W}$ **do**
- 4: Compute causal rolling mean, std, median on $\{x_{i-u+1}, \dots, x_i\}$ with minimum periods $\max(2, \lfloor u/3 \rfloor)$.
- 5: Compute causal rolling MAD on the same window: $\text{MAD}(S) = \text{median}(|S - \text{median}(S)|)$.
- 6: **end for**
- 7: **First differences:** $\Delta x_i \leftarrow x_i - x_{i-1}$, $\Delta^2 x_i \leftarrow x_i - x_{i-2}$.
- 8: **Percent change:** $\text{pct}_i \leftarrow (x_i - x_{i-1}) / (|x_{i-1}| + \epsilon)$ and map $\pm\infty \mapsto \text{NaN}$.
- 9: **Causal robust z-score:**

$$z_i \leftarrow \frac{x_i - \text{median}(x_{i-w_z+1:i})}{1.4826 \cdot \text{MAD}(x_{i-w_z+1:i})},$$

where the rolling MAD is computed causally; forward-fill the MAD for initial stability and apply a small floor if needed.

- 10: **Assemble** $\Phi(i)$ by concatenating rolling features, differences, percent change, and z_i .

- 11: **return** Φ
-

Algorithm 2 A2. ProxFuse proximity features (Leak-free, per series).

Require: Feature stream $\Phi(t) \in \mathbb{R}^p$ (time-ordered), split blocks (X_{tr}, X_{cf}, X_{ce}) , FFILL limit L (default 5), scaler and PCA (target dim $d = 3$), rolling-MAD window W , k_{MAD} , k_{NN} size K , HDBSCAN option with safe fallback.

Ensure: Proximity feature-augmented outputs: `prox_level_mu`, `prox_level_outlier`, `prox_velocity_mu`, `proto_dist_1`.

- 1: **Causal sanitization:** forward-fill $\Phi(\cdot)$ with limit L in time order; replace $\pm\infty$ with NaN; do not backfill.
- 2: **Train-median imputation (fit on X_{tr} only):** compute per-feature medians on X_{tr} ; fill NaNs in all blocks with these medians (all-NaN median $\rightarrow 0$).
- 3: **Train-only standardization:** fit scaler on X_{tr} ; transform all blocks.
- 4: **Train-only PCA embedding:** fit PCA on X_{tr} with $d' = \min(d, \#rows, \#features)$; transform all blocks to $Z(t) \in \mathbb{R}^{d'}$; pad with zeros to 3 dims $(\phi_1(t), \phi_2(t), \phi_3(t))$ if $d' < 3$.
- 5: Let $n_{tr} = |X_{tr}|$, and define $Z_{tr} = \{Z(t) : t \in X_{tr}\}$.
- 6: **Train-referenced KNN mean distance:** for each t , compute mean KNN distance from $Z(t)$ to Z_{tr} (for train points, use a train-internal KNN reference; for all points, use Z_{tr} as the reference set), yielding `knn_mean(t)`.
- 7: **Adaptive causal scale (rolling MAD on ϕ_1):** compute rolling $MAD_W(\phi_1(t))$ causally; set $\varepsilon_t \leftarrow \max(10^{-6}, k_{MAD} \cdot MAD_W(\phi_1(t)))$ with train-median fallback for NaNs.
- 8: **Level membership (distance-to-scale):** $\mu_{level}(t) \leftarrow 1 - \exp(-knn_mean(t)/(\varepsilon_t + \delta))$; apply train-only min-max scaling (fit on X_{tr}) and store as `prox_level_mu`.
- 9: **Level outlier cue (density):** if HDBSCAN is enabled and n_{tr} is large enough, fit HDBSCAN on Z_{tr} and take train outlier scores; otherwise fallback to `knn_mean` on train. For non-train points, use `knn_mean` as the outlier proxy. Apply train-only min-max scaling and store as `prox_level_outlier`.
- 10: **Velocity cue:** compute increments $\Delta Z(t) = Z(t) - Z(t-1)$ (zero at the first point); compute train-referenced KNN mean distances in ΔZ space; apply train-only min-max scaling and store as `prox_velocity_mu`.
- 11: **Prototype distance:** let `proto` = median(Z_{tr}) (component-wise); compute $d_1(t) = \|Z(t) - \text{proto}\|_2$; apply train-only min-max scaling and store as `proto_dist_1`.
- 12: **return** proximity-augmented series frame

Algorithm 3 A3. Leak-free supervised head, train-only isotonic calibration, exact top- k budget policy, metrics, and Intel triage.

Require: For each labeled series: feature set (including proximity features), labels $y(t) \in \{0, 1\}$ on $X_{cf} \cup X_{ce}$. Budget q (default 1%), bootstrap replicates N_{boot} (default 500). For Intel (unlabeled): core score or proximity-based operating score only.

Ensure: Deployed decision score and evaluation under strict out-of-sample protocol.

- 1: **Stable ids (recommended for mapping):** assign `__row_id__` over the full pooled table. Reset X_{cf} to assign `__cf_id__` and X_{ce} to assign `__ce_id__`. (Used to avoid timestamp collisions when mapping per-series scorers.)
- 2: **(Labeled) Supervised head (train-only):** train LightGBM on X_{tr} using the selected numeric feature columns (excluding `label`, `series_id`, `ids`). Obtain raw probabilities $\hat{p}_{cf}$ on X_{cf} and $\hat{p}_{ce}$ on X_{ce} .
- 3: **(Labeled) Probability calibration (calib_fit-only):** if y on X_{cf} contains both classes, fit isotonic regression g on $(\hat{p}_{cf}, y_{cf})$; otherwise set g to identity (single-class fallback). Define deployed score $p_b(t) \leftarrow g(\hat{p}(t))$ and apply unchanged to X_{ce} .
- 4: **(Operating point learned on X_{cf}):** $k_{fit} \leftarrow \lceil q \cdot |X_{cf}| \rceil$; select the indices of the top- k_{fit} values of p_b on X_{cf} (deterministic exact top- k); report τ_{fit} as the minimum selected score.
- 5: **(Evaluation on X_{ce}):** apply the same k_{fit} by selecting exact top- k_{fit} on X_{ce} . Compute ROC-AUC and PR-AUC on $p_b(t)$ over X_{ce} (when both classes are present), and compute $PR@q$ and $R@q$ from the top- k_{fit} alarm set.
- 6: **(Uncertainty):** compute CI95 via series-aware bootstrap: resample complete series with replacement, recompute pooled metrics, and report quantiles [0.025, 0.975]. Optionally compute paired Δ -CI95 on matched resamples.
- 7: **(Intel unlabeled triage):** disable the supervised head and do not claim supervised performance. Use the operating score and fit a label-free normalizer (ECDF/ChECDF) on X_{cf} ; transfer it unchanged to X_{ce} . Under the same budget q , select top- k_{fit} alarms on X_{ce} and report triage statistics (coverage, stability, concentration, and event characteristics), alongside representative classical scorers.
- 8: **return** Reported metrics (labeled) or triage statistics (Intel), all under strict leak-free transfer.

References

- [1] A. Chatterjee, B.S. Ahmed, IoT anomaly detection methods and applications: a survey, *Internet Things* 19 (2022) 100568. <https://doi.org/10.1016/j.iot.2022.100568>
- [2] C.C. Aggarwal, An introduction to outlier analysis, *Commun ACM* 65 (1) (2022) 80–90. <https://doi.org/10.1145/3439950>
- [3] A. Lavin, S. Ahmad, Evaluating real-time anomaly detection algorithms - the numenta anomaly benchmark, in: *Proc. IEEE 14th International Conference on Machine Learning and Applications (ICMLA)*, Miami, FL, USA, 2015, pp. 38–44. <https://doi.org/10.1109/ICMLA.2015.141>
- [4] S. Ahmad, A. Lavin, S. Purdy, Z. Agha, Unsupervised real-time anomaly detection for streaming data, *Neurocomputing* 262 (2017) 134–147. <https://doi.org/10.1016/j.neucom.2017.04.070>
- [5] Numenta, Numenta anomaly benchmark (NAB) repository, 2026 GitHub. Available at, <https://github.com/numenta/NAB>.
- [6] S. Schmidl, P. Wenig, T. Papenbrock, Anomaly detection in time series: a comprehensive evaluation, *Proc. VLDB Endowment* 15 (2022) 1779–1797. <https://doi.org/10.14778/3538598.3538602>

- [7] P. Bodík, W. Hong, C. Guestrin, S. Madden, M. Paskin, R. Thibaux, Intel Berkeley research lab sensor data, kaggle dataset 2026, available at: <https://www.kaggle.com/datasets/divyansh22/intel-berkeley-research-lab-sensor-data/data>.
- [8] M. Breunig, H.-P. Kriegel, R. Ng, J. Sander, LOF: identifying density-based local outliers, *SIGMOD Rec.* 29 (2) (2000) 93–104. <https://doi.org/10.1145/342009.335388>
- [9] B. Schölkopf, J.C. Platt, J. Shawe-Taylor, A.J. Smola, R.C. Williamson, Estimating the support of a high-dimensional distribution, *Neural Comput.* 13 (7) (2001) 1443–1471. <https://doi.org/10.1162/089976601750264965>
- [10] F.T. Liu, K.M. Ting, Z.-H. Zhou, Isolation forest. Eighth IEEE International Conference on Data Mining, 2008, 413–422. <https://doi.org/10.1109/ICDM.2008.17>
- [11] J. Li, H. Izakian, W. Pedrycz, I. Jamal, Clustering-based anomaly detection in multivariate time series data, *Appl. Soft Comput.* 100 (2021) 106919. <https://doi.org/10.1016/j.asoc.2020.106919>
- [12] P. Malhotra, A. Ramakrishnan, G. Anand, L. Vig, P. Agarwal, G. Shroff, LSTM-based encoder-decoder for multi-sensor anomaly detection, 2016, <https://arxiv.org/abs/1607.00148>.
- [13] M. Munir, S.A. Siddiqui, A. Dengel, S. Ahmed, DeepAnT: a deep learning approach for unsupervised anomaly detection in time series, *IEEE Access* 7 (2019) 1991–2005. <https://doi.org/10.1109/ACCESS.2018.2886457>
- [14] H. Xu, W. Chen, N. Zhao, Z. Li, J. Bu, Z. Li, Y. Liu, Y. Zhao, D. Pei, Y. Feng, J. Chen, Z. Wang, H. Qiao, Unsupervised anomaly detection via variational auto-encoder for seasonal KPIs in web applications, in: *Proceedings of the 2018 World Wide Web Conference (WWW '18)*, 2018, pp. 187–196. CHE. <https://doi.org/10.1145/3178876.3185996>
- [15] G. Li, J.J. Jung, Deep learning for anomaly detection in multivariate time series: approaches, applications, and challenges, *Inf. Fusion*, 91 2023 93–102. <https://doi.org/10.1016/j.inffus.2022.10.008>
- [16] X. Xia, X. Pan, N. Li, X. He, L. Ma, X. Zhang, N. Ding, GAN-based anomaly detection: a review, *Neurocomputing* 493 (2022) 497–535. <https://doi.org/10.1016/j.neucom.2021.12.093>
- [17] Y. Su, Y. Zhao, C. Niu, R. Liu, W. Sun, D. Pei, Robust anomaly detection for multivariate time series through stochastic recurrent neural network, in: *Proc. ACM SIGKDD Int. Conf. Knowl. Discov. Data Min.*, 2019, pp. 2828–2837. <https://doi.org/10.1145/3292500.3330672>
- [18] J. Audibert, P. Michiardi, F. Guyard, S. Marti, M.A. Zuluaga, USAD: unsupervised anomaly detection on multivariate time series, in: *Proc. ACM SIGKDD Int. Conf. Knowl. Discov. Data Min.*, 2020, pp. 3395–3404. <https://doi.org/10.1145/3394486.3403392>
- [19] J. Xu, H. Wu, J. Wang, M. Long, A. Transformer, Time series anomaly detection with association discrepancy, in: *Proc. Int. Conf. Learn. Represent. (ICLR)*, 2022. <https://openreview.net/forum?id=LzQQ89U1qm>.
- [20] Z. Chen, Z. Li, X. Chen, X. Chen, H. Fan, R. Hu, Rectifying inaccurate unsupervised learning for robust time series anomaly detection, *Inf. Sci.* 662 (2024) 120222. <https://doi.org/10.1016/j.ins.2024.120222>
- [21] R. Ma, Y. Ma, X. Liu, Time series anomaly detection via temporal relationship graphs and adaptive smoothing, *Appl. Soft Comput.* 179 (2025) 113298. <https://doi.org/10.1016/j.asoc.2025.113298>
- [22] J. Han, Z. Chen, D. Zhou, B. Hu, T. Xia, E. Pan, Unsupervised motion-based anomaly detection with graph attention networks for industrial robots labeling, *Eng. Appl. Artif. Intell.* 146 (2025) 110298. <https://doi.org/10.1016/j.engappai.2025.110298>
- [23] Z. Min, Q. Xiao, M. Abbas, D. Zhang, Retentive network-based time series anomaly detection in cyber-physical systems, *Eng. Appl. Artif. Intell.* 145 (2025) 110215. <https://doi.org/10.1016/j.engappai.2025.110215>
- [24] H.T. Truong, B.P. Ta, Q.A. Le, D.M. Nguyen, C.T. Le, H.X. Nguyen, H.T. Do, H.T. Nguyen, K.P. Tran, Light-weight federated learning-based anomaly detection for time-series data in industrial control systems, *Comput. Ind.* 140 (2022) 103692. <https://doi.org/10.1016/j.compind.2022.103692>
- [25] R. Li, H. Ma, R. Wang, H. Song, X. Zhou, L. Wang, H. Zhang, K. Zeng, C. Xia, Application of unsupervised learning methods based on video data for real-time anomaly detection in wire arc additive manufacturing, *J. Manuf. Process.* 143 (2025) 37–55. <https://doi.org/10.1016/j.jmapro.2025.03.113>
- [26] Y. Song, C. Song, Adaptive evolutionary multitask optimization based on anomaly detection transfer of multiple similar sources, *Expert Syst. Appl.* 283 (2025) 127599. <https://doi.org/10.1016/j.eswa.2025.127599>
- [27] S. Cellár, M. Santos, F. Alonso, E. Fabregas, G. Farias, Explainable anomaly detection in spacecraft telemetry, *Eng. Appl. Artif. Intell.* 133 (2024) 108083. <https://doi.org/10.1016/j.engappai.2024.108083>
- [28] F. Wang, Y. Jiang, R. Zhang, A. Wei, J. Xie, X. Pang, A survey of deep anomaly detection in multivariate time series: taxonomy, applications, and directions, *Sensors* 25 (1) (2025) 190. <https://doi.org/10.3390/s25010190>
- [29] K. Choi, J. Yi, C. Park, S. Yoon, Deep learning for anomaly detection in time-series data: review, analysis, and guidelines, *IEEE Access* 9 (2021) 120043–120065. <https://doi.org/10.1109/ACCESS.2021.3107975>
- [30] Y. Jia, X. Gu, J. Liu, Z. Huang, J. Zhou, Deep anomaly detection for time series: a survey, *Comput. Sci. Rev.* 58 (2025) 100787. <https://doi.org/10.1016/j.cosrev.2025.100787>
- [31] M. Chalapathy, S. Chawla, Deep learning for anomaly detection: a survey, *IEEE Trans. Knowl. Data Eng.* 34 (5) (2022) 2227–2246. <https://doi.org/10.1109/TKDE.2021.3079966>
- [32] J. Davis, M. Goadrich, The relationship between precision-recall and ROC curves, in: *Proc. Int. Conf. Mach. Learn. (ICML)*, 2006, pp. 233–240. <https://doi.org/10.1145/1143844.1143874>
- [33] T. Saito, M. Rehmsmeier, The precision-recall plot is more informative than the ROC plot when evaluating binary classifiers on imbalanced datasets, *PLoS ONE* 10 (3) (2015) 118432. <https://doi.org/10.1371/journal.pone.0118432>
- [34] P.J. Rousseeuw, C. Croux, Alternatives to the median absolute deviation, *J. Am. Stat. Assoc.* 88 (424) (1993) 1273–1283. <https://doi.org/10.1080/01621459.1993.10476408>
- [35] A. Niculescu-Mizil, R. Caruana, Predicting good probabilities with supervised learning, in: *Proc. Int. Conf. Mach. Learn. (ICML)*, 2005, pp. 625–632. <https://doi.org/10.1145/1102351.1102430>
- [36] C. Guo, G. Pleiss, Y. Sun, K.Q. Weinberger, On calibration of modern neural networks, *Proc. Int. Conf. Mach. Learn.* 70 (2017) 1321–1330. <http://proceedings.mlr.press/v70/guo17a.html>
- [37] S.M. Lundberg, S.-I. Lee, A unified approach to interpreting model predictions, *Adv. Neural Inf. Process. Syst. (NeurIPS)* 30, 2017. <https://proceedings.neurips.cc/paper/2017/file/8a20a8621978632d76c43dfd28b67767-Paper.pdf>
- [38] D. Lee, S. Malacarne, E. Aune, Explainable time series anomaly detection using masked latent generative modeling, *Pattern Recognit.* 156 (2024) 110826. <https://doi.org/10.1016/j.patcog.2024.110826>
- [39] I. Ferrogli, G. Saveri, L. Nenzi, L. Bortolussi, ECATS: explainable-by-design concept-based anomaly detection for time series, In T.R. Besold, A. d'Avila Garcez, E. Jimenez-Ruiz, R. Confalonieri, P. Madhyastha, B. Wagner (Eds.), *Neural-Symbolic Learning and Reasoning (NeSy 2024)*, Lect. Notes Comput. Sci., 14980 of Cham, Springer, 2024. https://doi.org/10.1007/978-3-031-71170-1_16
- [40] NetManAIOps, OmniAnomaly: unsupervised and semi-supervised anomaly detection for time series, github repository, 2025. Available at: <https://github.com/NetManAIOps/OmniAnomaly.git>
- [41] Y. Chen, E. Keogh, B. Hu, N. Begum, A. Bagnall, A. Mueen, G. Batista, The UCR time series classification archive, 2015. Available at: https://www.cs.ucr.edu/~eamonn/time_series_data/.
- [42] Finloop, usad-on-ucr-data: USAD model on UCR Time Series Anomaly Archive, GitHub repository, 2025, Available at: <https://github.com/finloop/usad-on-ucr-data>.
- [43] T. Fawcett, An introduction to ROC analysis, *Pattern Recognit. Lett.* 27 (8) (2006) 861–874. <https://doi.org/10.1016/j.patrec.2005.10.010>